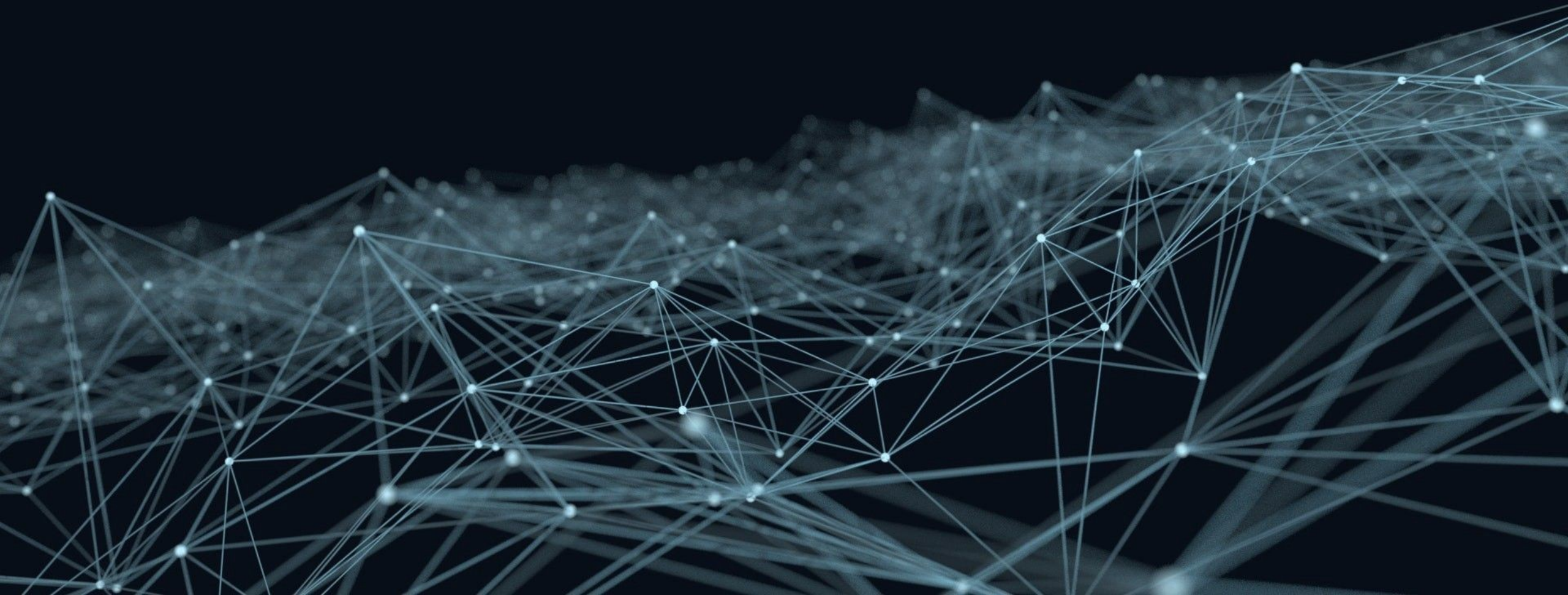
Java Full Stack Development

5. React and Security



TEKsystems Global Services

WORKFORCE DEVELOPMENT

Module Topics

- Routing and Navigation
- OAuth2/OIDC
- Pure-SPA Authentication
- BFF Pattern with Spring Boot

Routing and Navigation

Routing and Navigation

- Routing in a SPA
  - Users expect URLs that change as they navigate
  - Users expect the back button to work
  - Users expect to bookmark and share specific pages
  - Users expect to deep-link directly to a page
- An SPA loads one HTML document
  - None of this happens automatically

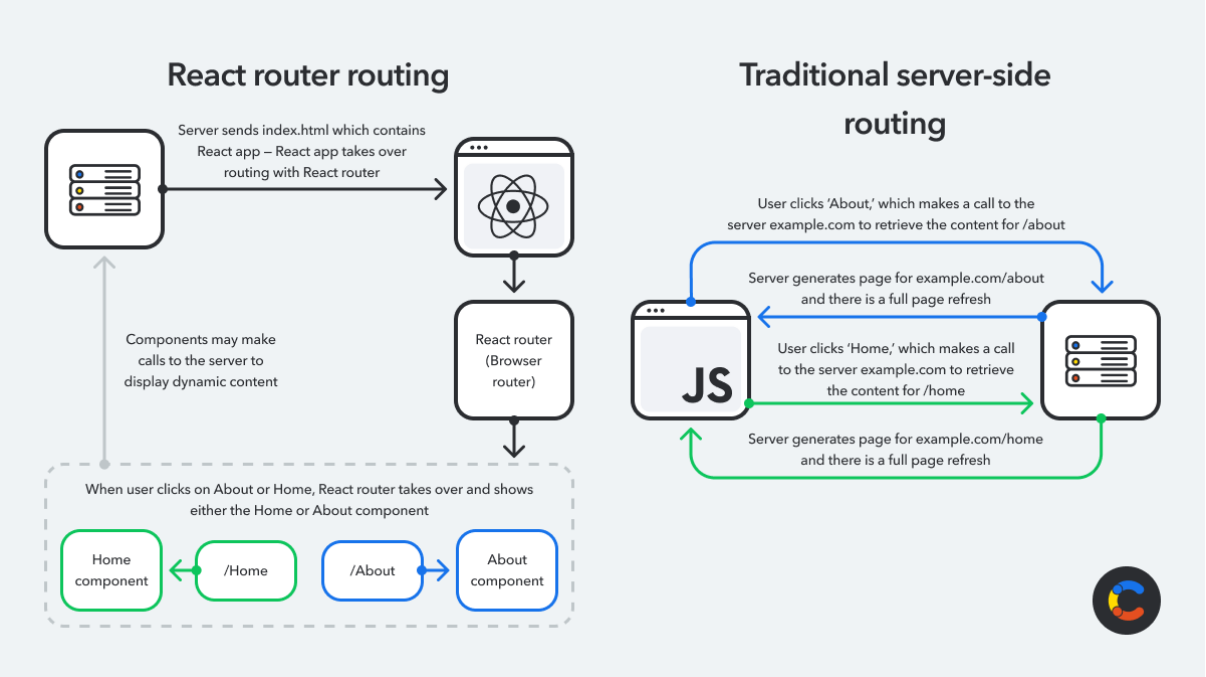


Image Credit: https://www.contentful.com/blog/react-routing/

Routing and Navigation

- Without routing
  - An SPA is one URL forever
    - The user navigates, the screen changes, but the address bar still says https://app.example.com/
    - The back button does nothing
    - Bookmarks always land on the home page
    - Deep-linking from an email straight to "the user's invoice page" is impossible
  - This is unacceptable for any real application.
    - Users have decades of intuition about how URLs work
    - They expect them to be meaningful and stable.

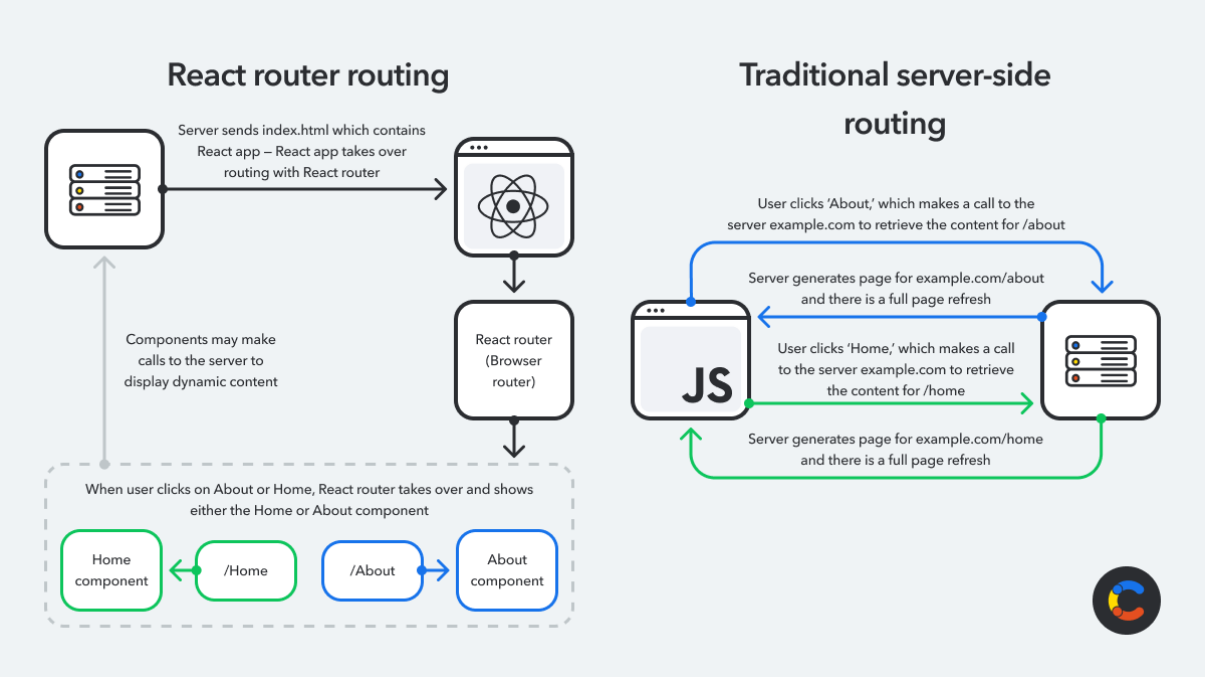


Image Credit: https://www.contentful.com/blog/react-routing/

Routing and Navigation

- Without routing
  - Routing makes a single-page application behave like a multi-page application from the user's perspective
    - The URL changes as they navigate
    - The back button works
    - They can bookmark /users/42 and come back to it
  - From the browser's perspective, it's all one HTML document
  - From the user's perspective, it's many pages

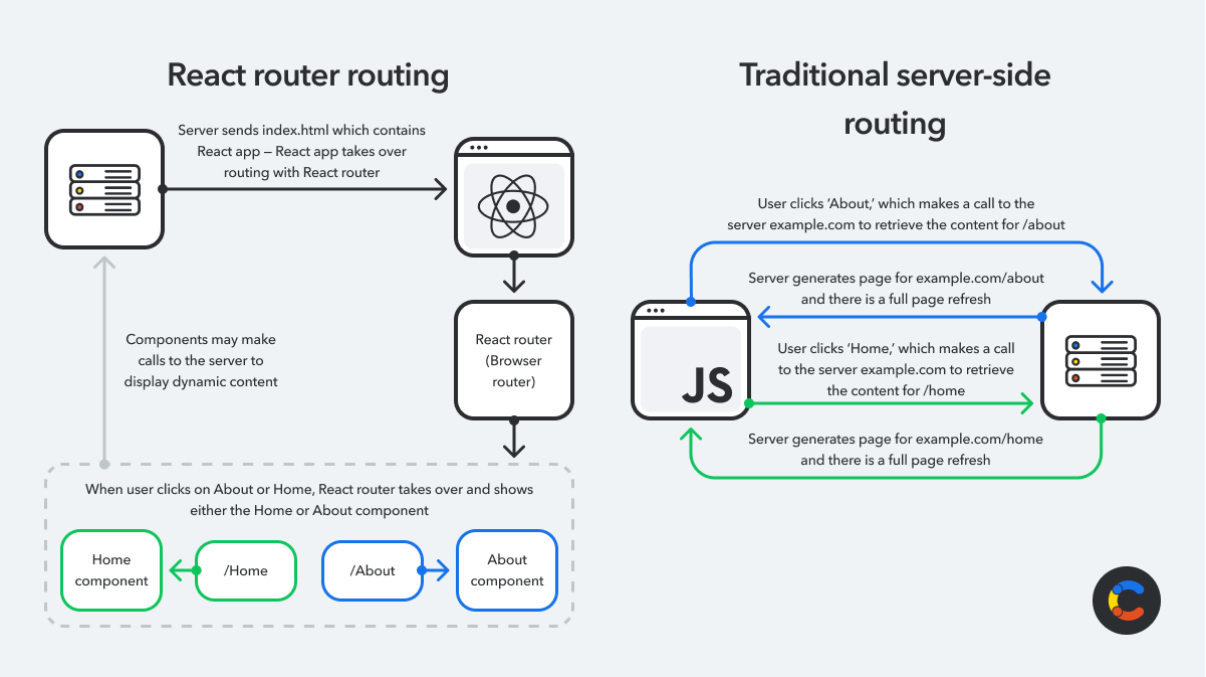


Image Credit: https://www.contentful.com/blog/react-routing/

Client-Side Routing

- The history API
  - Browsers expose history.pushState(state, title, url) to change the URL without reloading
  - React Router intercepts clicks on <Link> and calls pushState instead of letting the browser navigate
  - The URL changes, the back/forward buttons remember it, but no HTTP request is made
  - React Router reads the current URL and renders the matching component
  - The browser thinks it's on a new page; nothing was actually fetched

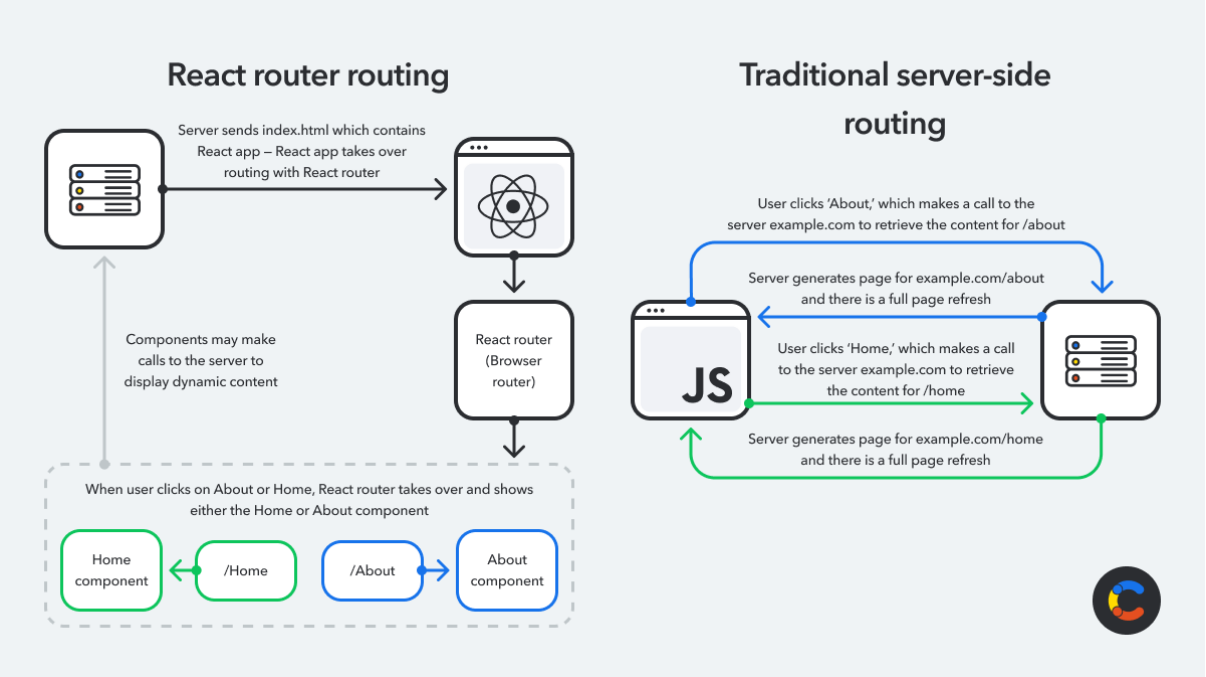


Image Credit: https://www.contentful.com/blog/react-routing/

React Router Basics

- Setting up React Router
  - <BrowserRouter> wraps the app, plugs into the History API
  - <Routes> is the matcher
    - Picks one route based on the URL
  - <Route> declares a path-to-component mapping
  - :id marks a route parameter captured from the URL
  - * is the catch-all, used for 404s

```
npm install react-router-dom
```

```
import { BrowserRouter, Routes, Route } from 'react-router-dom';

function App() {
  return (
    <BrowserRouter>
      <Routes>
        <Route path="/" element={<HomePage />} />
        <Route path="/users" element={<UserListPage />} />
        <Route path="/users/:id" element={<UserDetailPage />} />
        <Route path="*" element={<NotFoundPage />} />
      </Routes>
    </BrowserRouter>
  );
}
```

React Router Basics

- Setting up React Router
  - <BrowserRouter> is the integration point with the browser
    - It listens to the History API, provides routing context to all descendants, and manages the URL
    - Wrap it once at the top of your app.
  - <Routes> is the route matcher
    - At any moment, exactly one of its children matches the current URL, and that one renders
    - The others render nothing.
  - <Route> declares a path and what to render when it matches
    - The element prop is the JSX to render
    - Typically a page-level component

```
npm install react-router-dom
```

```
import { BrowserRouter, Routes, Route } from 'react-router-dom';

function App() {
  return (
    <BrowserRouter>
      <Routes>
        <Route path="/" element={<HomePage />} />
        <Route path="/users" element={<UserListPage />} />
        <Route path="/users/:id" element={<UserDetailPage />} />
        <Route path="*" element={<NotFoundPage />} />
      </Routes>
    </BrowserRouter>
  );
}
```

React Router Basics

- Path syntax
  - /users matches exactly /user
  - /users/:id matches /users/anything and exposes id as a parameter.
  - * matches anything that no other route matched
  - React Router uses a ranking algorithm to pick the most specific match
    - Not just the first one declared
    - So /users/:id and /users/me can both be defined
    - Navigating to /users/me will pick the literal me route, not the parameterized one

```
npm install react-router-dom
```

```
import { BrowserRouter, Routes, Route } from 'react-router-dom';

function App() {
  return (
    <BrowserRouter>
      <Routes>
        <Route path="/" element={<HomePage />} />
        <Route path="/users" element={<UserListPage />} />
        <Route path="/users/:id" element={<UserDetailPage />} />
        <Route path="*" element={<NotFoundPage />} />
      </Routes>
    </BrowserRouter>
  );
}
```

Navigation

- Navigating without reloading
  - <Link> renders an <a> tag but intercepts the click for client-side navigation
    - Never use a plain <a href> for in-app navigation
    - It triggers a full reload
  - <NavLink> is <Link> with active-state awareness for menu highlighting
    - to accepts a path string or a location object
  - Replace prop swaps the current history entry instead of adding a new one

```jsx
import { Link, NavLink } from 'react-router-dom';

function Navigation() {
  return (
    <nav>
      <Link to="/">Home</Link>
      <Link to="/users">Users</Link>

      {/* NavLink is Link + automatic styling for the active route */}
      <NavLink
        to="/dashboard"
        className={({ isActive }) => isActive ? 'nav-active' : ''}
      >
        Dashboard
      </NavLink>
    </nav>
  );
}
```

Navigation

- Navigating without reloading
  - <Link> is what you use instead of <a> for any in-app navigation
    - It produces an <a> tag in the DOM
    - it's still a real link: right-click, copy link, open in new tab all work
    - but the click handler intercepts the default browser behavior and routes via React Router.
  - The rule:
    - <Link> for in-app navigation,
    - <a> for external links
    - If the user clicks <a href="/users">, the browser does a full page reload
    - Then the React app re-initializes from scratch, and any in-memory state is lost.
    - With <Link to="/users">, React Router handles the URL change and just renders the new component and the rest of the app stays alive

```jsx
import { Link, NavLink } from 'react-router-dom';

function Navigation() {
  return (
    <nav>
      <Link to="/">Home</Link>
      <Link to="/users">Users</Link>

      {/* NavLink is Link + automatic styling for the active route */}
      <NavLink
        to="/dashboard"
        className={({ isActive }) => isActive ? 'nav-active' : ''}
      >
        Dashboard
      </NavLink>
    </nav>
  );
}
```

Navigation

- Navigating without reloading
  - <NavLink> is the variant for navigation menus
    - It automatically knows whether its target route is currently active, and you can style it accordingly
    - The className prop accepts a function that receives { isActive, isPending } and returns a string
    - Common pattern for highlighting the current section in a sidebar or nav bar.
  - The replace prop is occasionally useful
    - By default, navigation pushes a new history entry
    - With replace, the current entry is overwritten

```jsx
import { Link, NavLink } from 'react-router-dom';

function Navigation() {
  return (
    <nav>
      <Link to="/">Home</Link>
      <Link to="/users">Users</Link>

      {/* NavLink is Link + automatic styling for the active route */}
      <NavLink
        to="/dashboard"
        className={({ isActive }) => isActive ? 'nav-active' : ''}
      >
        Dashboard
      </NavLink>
    </nav>
  );
}
```

Reading the URL

- Hooks for reading the URL
  - UseParams
    - Gives the values captured by :name placeholders in the route definition
    - If the route is /users/:id and the URL is /users/42
    - useParams() returns { id: '42' }
    - Note the value is always a string
    - Convert with Number(id) if you need it as a number.
  - UseSearchParams
    - Equivalent of URLSearchParams for the query string
    - The first element is the params object (read-only)
    - The second is a setter that updates the UR
    - Useful for filter/search/tab state that should survive bookmark and reload.

```
import { useParams, useSearchParams, useLocation } from 'react-router-dom';

function UserDetailPage() {
  const { id } = useParams<{ id: string }>();              // /users/:id
  const [searchParams] = useSearchParams();                // ?tab=settings
  const location = useLocation();                          // full location obj

  const tab = searchParams.get('tab') ?? 'overview';

  return <div>User {id}, viewing {tab}</div>;
}
```

Reading the URL

- Hooks for reading the URL
  - useLocation gives you everything
    - The pathname
    - The search string, the hash, and any state object passed via navigation
    - The state slot is useful for transient data
    - For example, when redirecting to login, you can pass the originally-requested URL in the state, then read it after login completes.

```
import { useParams, useSearchParams, useLocation } from 'react-router-dom';

function UserDetailPage() {
  const { id } = useParams<{ id: string }>();              // /users/:id
  const [searchParams] = useSearchParams();                // ?tab=settings
  const location = useLocation();                          // full location obj

  const tab = searchParams.get('tab') ?? 'overview';

  return <div>User {id}, viewing {tab}</div>;
}
```

Programmatic Navigation

- useNavigate
  - useNavigate returns a function for triggering navigation from code
    - Pass a path to navigate forward
    - Pass a number for relative back/forward
  - Replace true overwrites the current history entry
  - Use after form submissions, async operations, or conditional flows
  - Don't navigate during render
    - Only in event handlers or effects

```tsx
import { useNavigate } from 'react-router-dom';

function LoginForm() {
  const navigate = useNavigate();

  async function handleLogin(credentials: Credentials) {
    await login(credentials);
    navigate('/dashboard');                  // forward to dashboard
  }

  function handleCancel() {
    navigate(-1);                            // back, like the browser button
  }

  function handleAfterDelete() {
    navigate('/users', { replace: true });   // replace history entry
  }
}
```

Nested Routes and Layouts

- Nested routes and shared layouts
  - A <Route> with no path is a layout route
  - The layout wraps all its child routes with shared UI
  - <Outlet> is the placeholder where the matched child renders
  - Login and other "no chrome" routes can sit outside the layout
  - Layouts can nest arbitrarily deep

```
<Routes>
  <Route element={<AppLayout />}>
    <Route path="/" element={<HomePage />} />
    <Route path="/dashboard" element={<DashboardPage />} />
    <Route path="/users" element={<UserListPage />} />
  </Route>
  <Route path="/login" element={<LoginPage />} />
</Routes>


function AppLayout() {
  return (
    <div>
      <Header />
      <Sidebar />
      <main>
        <Outlet />     {/* nested route renders here */}
      </main>
    </div>
  );
}
```

Nested Routes and Layouts

- Shared UI
  - Most apps have shared UI
    - A header, a sidebar, a footer
    - Should appear on most pages but not all
    - Nested routes handle this cleanly.
  - A layout route is a route with no path
  - Declares "everything inside me shares this wrapping"
  - The layout component renders shared chrome and an <Outlet> placeholder element
    - Outlet = "render the matched child route here".
  - The layout route wraps Home, Dashboard, and Users
    - All three render inside <AppLayout>, with header and sidebar visible.
  - Login is a sibling of the layout route, not a child
    - Renders without the app chrome just the login form on a clean page.

```
<Routes>
  <Route element={<AppLayout />}>
    <Route path="/" element={<HomePage />} />
    <Route path="/dashboard" element={<DashboardPage />} />
    <Route path="/users" element={<UserListPage />} />
  </Route>
  <Route path="/login" element={<LoginPage />} />
</Routes>
```

```
function AppLayout() {
  return (
    <div>
      <Header />
      <Sidebar />
      <main>
        <Outlet />     {/* nested route renders here */}
      </main>
    </div>
  );
}
```

Route Parameters

- Route parameters are always strings (URL segments)
  - Type them with a generic on useParams for autocomplete and safety
  - Multiple parameters in one route are fine
  - Convert to numbers with Number(...) if needed
  - Always handle the undefined case — TypeScript marks them optional

```tsx
// Route definition
<Route path="/users/:userId/posts/:postId" element={<PostDetail />} />

// Component
import { useParams } from 'react-router-dom';

type PostParams = {
  userId: string;
  postId: string;
};

function PostDetail() {
  const { userId, postId } = useParams<PostParams>();
  // userId and postId are strings

  if (!userId || !postId) return null;  // params can be undefined

  return <div>Post {postId} by user {userId}</div>;
}
```

Common Routing Pitfalls

- Common errors
  - Using <a href> instead of <Link>
    - Full page reload, lost state
  - Forgetting the * route
    - Bad URLs render nothing, no 404
  - Not configuring the static host for SPA fallback
    - Direct links 404 in production
  - Calling navigate during render
    - infinite loops, warnings
  - Not treating route params as strings
  - Putting sensitive data in the URL
    - Params are public

```
// Route definition
<Route path="/users/:userId/posts/:postId" element={<PostDetail />} />

// Component
import { useParams } from 'react-router-dom';

type PostParams = {
  userId: string;
  postId: string;
};

function PostDetail() {
  const { userId, postId } = useParams<PostParams>();
  // userId and postId are strings

  if (!userId || !postId) return null;  // params can be undefined

  return <div>Post {postId} by user {userId}</div>;
}
```

OAuth2/OIDC

The Four Roles

- Recap from week 2
  - Resource Owner
    - The user
  - Client
    - The React SPA running in the user's browser
  - Authorization Server
    - Issues tokens after authenticating the user (the mock server from the prior module)
  - Resource Server
    - The API holding the protected data (the Spring Boot REST API)
  - The roles are the same
    - What changes is where the client lives

Role	Responsibility in the Protocol
Resource Owner	The entity that owns the protected resources and has the authority to grant access to them. In interactive flows this is a human user who has an account with the authorisation server. In machine-to-machine flows — where no user is present — the resource owner is the system or service itself.
Client	The application that wants access to protected resources on behalf of the resource owner. The client requests tokens from the authorisation server and presents those tokens to the resource server. It never handles the resource owner's credentials.
Authorisation Server (AS)	The trusted authority that authenticates the resource owner, obtains consent where required, and issues tokens. The AS owns the complete token lifecycle: it mints tokens, signs them with its private key, and publishes the public keys needed to verify them. Examples: Keycloak, Auth0, Azure Active Directory, Okta.
Resource Server (RS)	The API that holds the protected resources the client wants to access. The RS validates incoming tokens on every request — verifying the signature, checking expiry and issuer, and enforcing scope requirements — before allowing or denying the operation.

Public vs. Confidential Client

- Public vs. confidential client
  - Confidential client
    - Runs in a controlled environment, can keep secrets
    - Server-side web apps, backend services, mobile apps with secure storage
    - Has a client_secret that proves its identity to the authorization server
  - Public client
    - Runs where users (and attackers) can inspect it, cannot keep secrets
    - Browser SPAs, native apps, single-binary CLIs
    - Anything in the bundle is downloadable, decompilable, or visible in DevTools
  - Every browser SPA is a public client
    - It's the nature of the deployment model.
    - Public clients require different protections than confidential clients

Role	Responsibility in the Protocol
Resource Owner	The entity that owns the protected resources and has the authority to grant access to them. In interactive flows this is a human user who has an account with the authorisation server. In machine-to-machine flows — where no user is present — the resource owner is the system or service itself.
Client	The application that wants access to protected resources on behalf of the resource owner. The client requests tokens from the authorisation server and presents those tokens to the resource server. It never handles the resource owner's credentials.
Authorisation Server (AS)	The trusted authority that authenticates the resource owner, obtains consent where required, and issues tokens. The AS owns the complete token lifecycle: it mints tokens, signs them with its private key, and publishes the public keys needed to verify them. Examples: Keycloak, Auth0, Azure Active Directory, Okta.
Resource Server (RS)	The API that holds the protected resources the client wants to access. The RS validates incoming tokens on every request — verifying the signature, checking expiry and issuer, and enforcing scope requirements — before allowing or denying the operation.

Public vs. Confidential Client

- The OAuth2 specification (RFC 6749)
  - Explicitly defines these two categories
  - Confidential client, eg. Spring Boot web app
    - Has a client_secret stored in a config file the user never sees
    - Can authenticate itself to the authorization server using that secret
    - Can hold long-lived tokens in server memory, where the user can't reach them
    - Runs in an environment where the operator controls what code executes

Role	Responsibility in the Protocol
Resource Owner	The entity that owns the protected resources and has the authority to grant access to them. In interactive flows this is a human user who has an account with the authorisation server. In machine-to-machine flows — where no user is present — the resource owner is the system or service itself.
Client	The application that wants access to protected resources on behalf of the resource owner. The client requests tokens from the authorisation server and presents those tokens to the resource server. It never handles the resource owner's credentials.
Authorisation Server (AS)	The trusted authority that authenticates the resource owner, obtains consent where required, and issues tokens. The AS owns the complete token lifecycle: it mints tokens, signs them with its private key, and publishes the public keys needed to verify them. Examples: Keycloak, Auth0, Azure Active Directory, Okta.
Resource Server (RS)	The API that holds the protected resources the client wants to access. The RS validates incoming tokens on every request — verifying the signature, checking expiry and issuer, and enforcing scope requirements — before allowing or denying the operation.

Public vs. Confidential Client

- The OAuth2 specification (RFC 6749)
  - A public client, like React SPA
    - Cannot keep a client secret
    - Any "secret" baked into the bundle is downloaded by every user and visible in DevTools.
    - Cannot prove its identity to the authorization server with a shared secret
  - Holds tokens in an environment the user controls
    - DevTools, browser extensions, any script running on the page
  - Runs in an environment where the user (or anything that's compromised the user) can do almost anything

Role	Responsibility in the Protocol
Resource Owner	The entity that owns the protected resources and has the authority to grant access to them. In interactive flows this is a human user who has an account with the authorisation server. In machine-to-machine flows — where no user is present — the resource owner is the system or service itself.
Client	The application that wants access to protected resources on behalf of the resource owner. The client requests tokens from the authorisation server and presents those tokens to the resource server. It never handles the resource owner's credentials.
Authorisation Server (AS)	The trusted authority that authenticates the resource owner, obtains consent where required, and issues tokens. The AS owns the complete token lifecycle: it mints tokens, signs them with its private key, and publishes the public keys needed to verify them. Examples: Keycloak, Auth0, Azure Active Directory, Okta.
Resource Server (RS)	The API that holds the protected resources the client wants to access. The RS validates incoming tokens on every request — verifying the signature, checking expiry and issuer, and enforcing scope requirements — before allowing or denying the operation.

Implicit Flow

- The deprecated implicit flow
  - Original OAuth2 flow for browser SPAs
    - Authorization server returned the access token directly in the URL fragment
  - No code exchange step, no PKCE, no client authentication
  - Deprecated by the OAuth in 2019
  - Removed entirely from OAuth 2.1

```
https://app.example.com/
callback#access_token=abc123&token_type=Bearer&expires_in=3600
```

Grant Type	User Present	Client Type	Status
Authorization Code + PKCE	Yes	Public or confidential	Current standard — use for all interactive flows
Client Credentials	No	Confidential only	Current standard — use for service-to-service
Refresh Token	No (after initial auth)	Public or confidential	Not standalone — used alongside Authorization Code
Device Code	Yes (on a second device)	Public	Limited input devices — not covered in this course
Implicit	Yes	Public	**Deprecated — do not use**
Resource Owner Password	Yes	Confidential	**Deprecated — do not use**

Implicit Flow

- Why this was abandoned
  - Tokens leaked through browser history, server logs, and referrer headers
    - Even though the fragment isn't sent to servers, it appears in the address bar, can be copied, and can leak through analytics scripts.
  - No way to refresh
    - No refresh token, so users had to silently re-authenticate via hidden iframes
  - No client authentication or PKCE-style binding
    - Anyone who intercepted the redirect could steal the token.
  - Token leakage to third-party scripts
    - Any script on the redirect page (analytics, ads, error trackers) could read the URL fragment

Grant Type	User Present	Client Type	Status
Authorization Code + PKCE	Yes	Public or confidential	Current standard — use for all interactive flows
Client Credentials	No	Confidential only	Current standard — use for service-to-service
Refresh Token	No (after initial auth)	Public or confidential	Not standalone — used alongside Authorization Code
Device Code	Yes (on a second device)	Public	Limited input devices — not covered in this course
Implicit	Yes	Public	**Deprecated — do not use**
Resource Owner Password	Yes	Confidential	**Deprecated — do not use**

PKCE for Browsers

- Authorization code + PKCE
  - Authorization Code flow from week 2
    - The browser generates a one-time random code_verifier per login attempt
  - Sends a code_challenge when redirecting to the auth server
    - code_challenge is a hash of the verifier string
  - After login
    - Exchanges the authorization code for tokens, including the original verifier
    - The auth server checks that hash(verifier) == challenge
    - Proves the same client started and finished the flow
  - Replaces the missing client secret for public clients

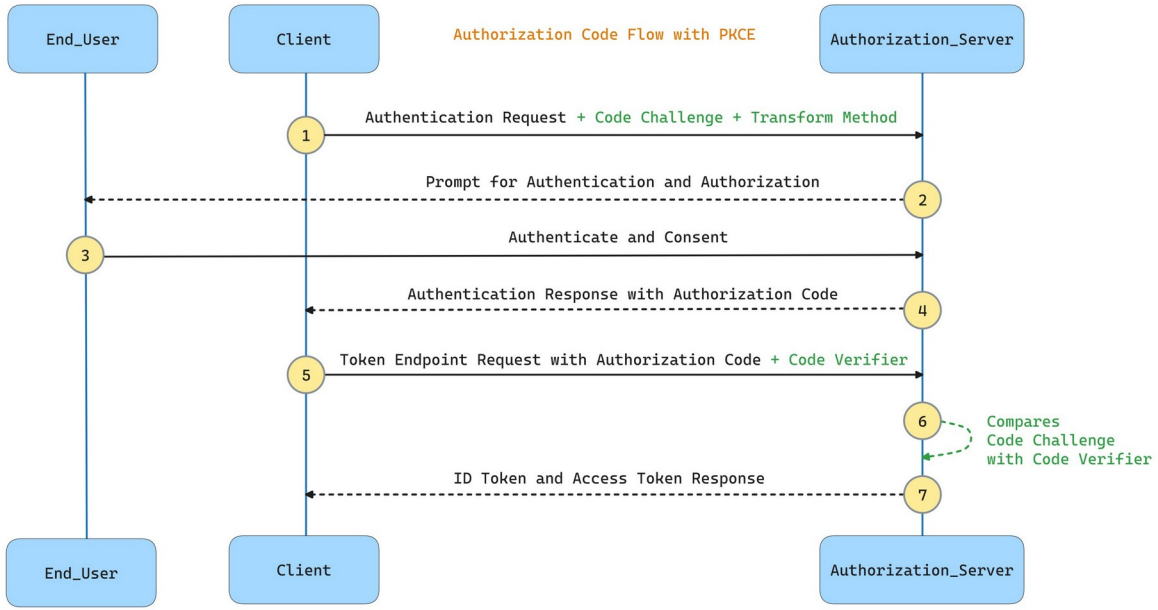


Image Credit: https://auth0.com/blog/pkce-in-web-applications-with-spring-security/

PKCE for Browsers

- PKCE adds a per-request secret that
  - Generated randomly at the start of each login attempt.
    - Stays in the browser the whole time, typically in sessionStorage
    - Sent to the auth server only at the end, during the code-for-token exchange.
    - Is bound to the original authorization request via the challenge.
  - The result
    - Even if an attacker intercepts the authorization code
    - They don't have the verifier, so they can't exchange the code for tokens
    - PKCE turns a one-step interception attack into a two-step attack that requires compromising the browser session itself

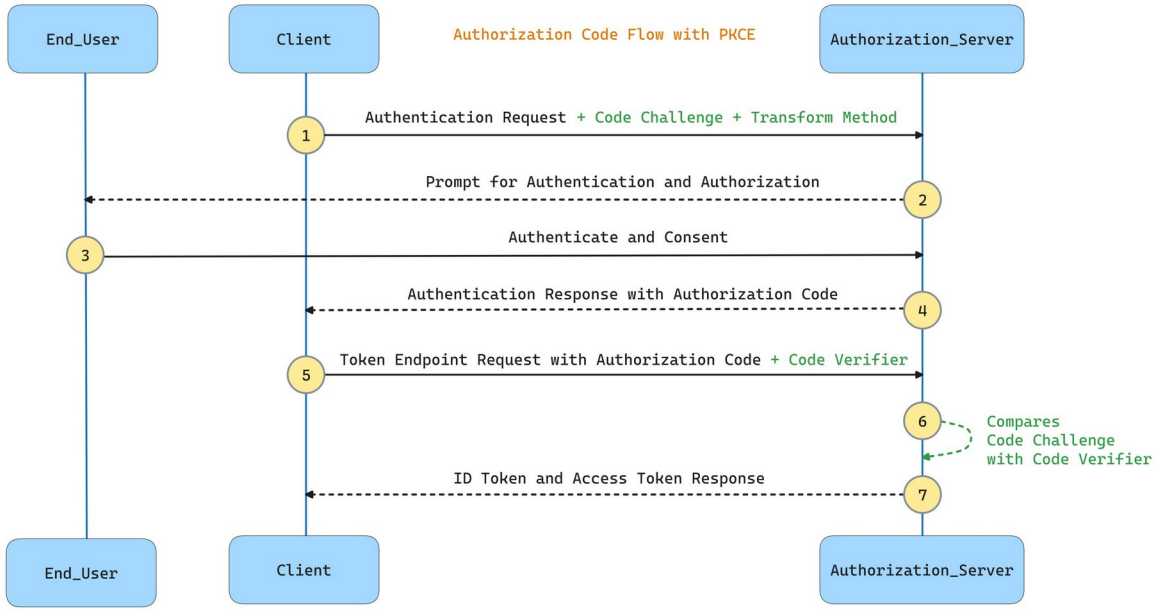


Image Credit: https://auth0.com/blog/pkce-in-web-applications-with-spring-security/

OIDC on Top of OAuth2

- Authentication, not just authorization
  - OAuth2 is about authorization
    - "This token can access these resources"
  - OIDC adds authentication
    - "This is the user, identity verified"
    - Adds an ID token. a signed JWT proving who the user is
    - Adds a nonce parameter, protects against ID token replay
    - Adds a userinfo endpoint, for fetching profile data after login
  - Most modern auth systems use OIDC, not raw OAuth2

Claim	Description	Specified by	Mandatory in ID Token
iss	Issuer of the token	OIDC / JWT	yes
sub	Subject of the token	OIDC / JWT	yes
aud	Recipient that the token is intended for	OIDC / JWT	yes
exp	Expiration time	OIDC / JWT	yes
iat	Time at which the token was issued	OIDC / JWT	yes
auth_time	Time when the end user authentication occurred	OIDC	no
nonce	Value to associate a client session with the token	OIDC	no
acr	Authentication Context Class Reference	OIDC	no
amr	Identifiers for authentication methods used in the authentication	OIDC	no
azp	OAuth 2.0 Client ID of the party to which the token was issued	OIDC	no
nbf	Time before which the token must not be accepted for processing	JWT	no
jti	Unique identifier for the token	JWT	no

Image Credit: https://www.vincit.com/blog/oauth-2-0-and-oidc-what-should-i-know-about-tokens

Browser Threat Model

- What can go wrong in the browser
  - Cross-site scripting (XSS)
    - Attacker runs JavaScript on your origin
  - Cross-site request forgery (CSRF)
    - Attacker triggers authenticated requests from a victim's browser
  - Open redirects
    - Attacker manipulates the redirect URI to steal codes/tokens

Image Credit: https://www.vincit.com/blog/oauth-2-0-and-oidc-what-should-i-know-about-tokens

Browser Threat Model

- What can go wrong in the browser
  - Token leakage
    - Through URLs, logs, analytics scripts, browser extensions
  - Network attacks
    - Public Wi-Fi, intercepting proxies (mitigated by HTTPS, but not fully)
  - Browser extensions
    - Can read every page the user visits

Image Credit: https://www.vincit.com/blog/oauth-2-0-and-oidc-what-should-i-know-about-tokens

Browser Threat Model

- XSS and the SPA token problem
  - XSS = attacker-controlled JavaScript runs on your origin
    - Injected JavaScript can do anything your code can do
    - If your code can read tokens from localStorage, the attacker's code can too
    - If your code can call /api/transfer with a bearer token, so can the attacker's code
    - One injection point enough for a full account takeover
  - No client-side storage mechanism is XSS-resistant if the running JavaScript is compromised

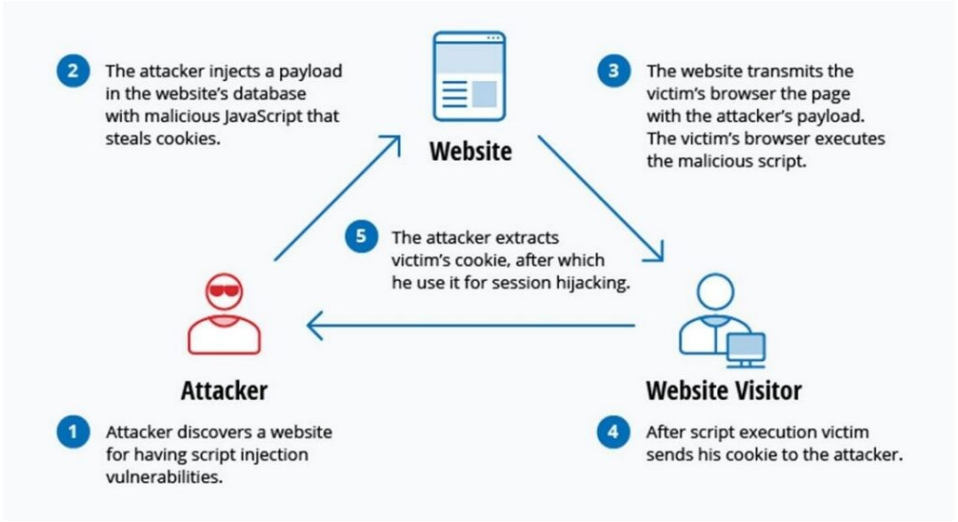


Image Credit: https://www.hackercoolmagazine.com/cross-site-scripting-for-beginners/

Browser Threat Model

- XSS mitigations
  - Content Security Policy (CSP)
    - Restricts what scripts the browser will execute
  - Subresource Integrity (SRI)
    - Verifies CDN-loaded scripts haven't been tampered with
  - Sanitizing user input
    - Preventing the XSS in the first place
  - Dependency scanning
    - Catching malicious npm packages
  - All of these reduce risk
    - None of them eliminate it
  - A single missed mitigation can result in ex-filtration of tokens

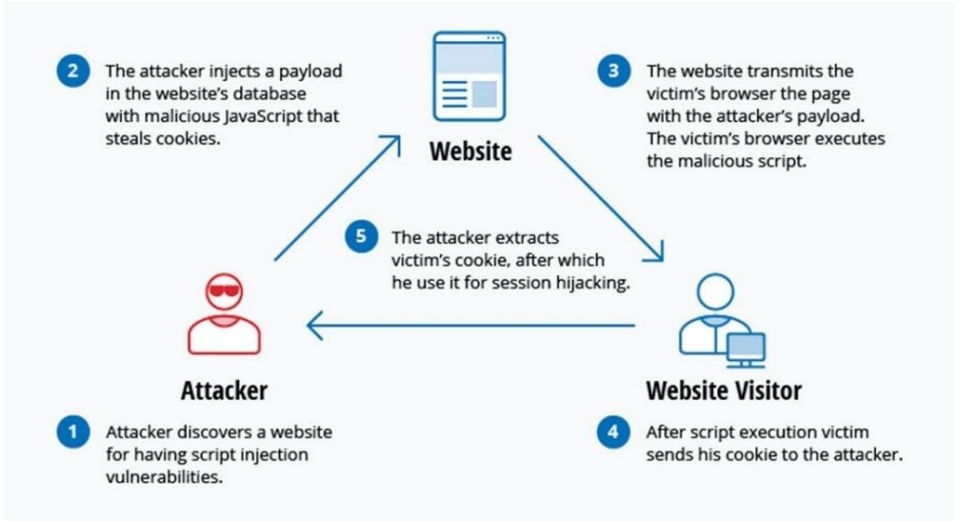


Image Credit: https://www.hackercoolmagazine.com/cross-site-scripting-for-beginners/

BFF

- Pure SPA
  - Tokens live in JavaScript; SPA is the OAuth client
  - Simplest deployment (just static files)
  - Most exposed to XSS
    - Token theft is full account compromise
  - Acceptable for low-risk apps with strong XSS defences
- Backend-for-Frontend
  - Tokens live on a backend you control; browser holds an HttpOnly session cookie
  - Requires a backend service like Spring Boot
  - XSS becomes session-bounded, not catastrophic
  - The current OAuth working group recommendation for sensitive applications

Concern	Pure SPA	BFF
Where do tokens live?	Browser JavaScript	Server-side session
What can XSS steal?	Tokens (full impersonation)	Nothing usable beyond active session
OAuth client type	Public	Confidential
Backend required	No	Yes
Same-origin deployment	Optional	Natural
CORS concerns	Yes (cross-origin API)	No (same-origin BFF)
Complexity	Lower	Moderate

BFF

- Industry trends
  - OAuth working group
    - Current best-practice drafts recommend BFF for sensitive applications
  - Major SaaS providers
    - Offer BFF-style integrations as the recommended path
  - Banks, healthcare, regulated industries
    - Moving to BFF for new SPA development
  - Pure SPA still common
    - For lower-risk applications and rapid development

Concern	Pure SPA	BFF
Where do tokens live?	Browser JavaScript	Server-side session
What can XSS steal?	Tokens (full impersonation)	Nothing usable beyond active session
OAuth client type	Public	Confidential
Backend required	No	Yes
Same-origin deployment	Optional	Natural
CORS concerns	Yes (cross-origin API)	No (same-origin BFF)
Complexity	Lower	Moderate

Pure-SPA Authentication

SPA

- Library choice
  - Don't roll your own
    - PKCE has subtle requirements, refresh has race conditions, logout has edge cases
  - oidc-client-ts
    - The underlying TypeScript OAuth/OIDC library, framework-agnostic
  - react-oidc-context
    - A thin React wrapper providing context and hooks
  - Both maintained by the OIDC client community, widely used in production
  - Alternatives
    - Auth0 SDK, MSAL (Microsoft), Okta SDK
    - For a generic OIDC provider (like our mock auth server), react-oidc-context is the default

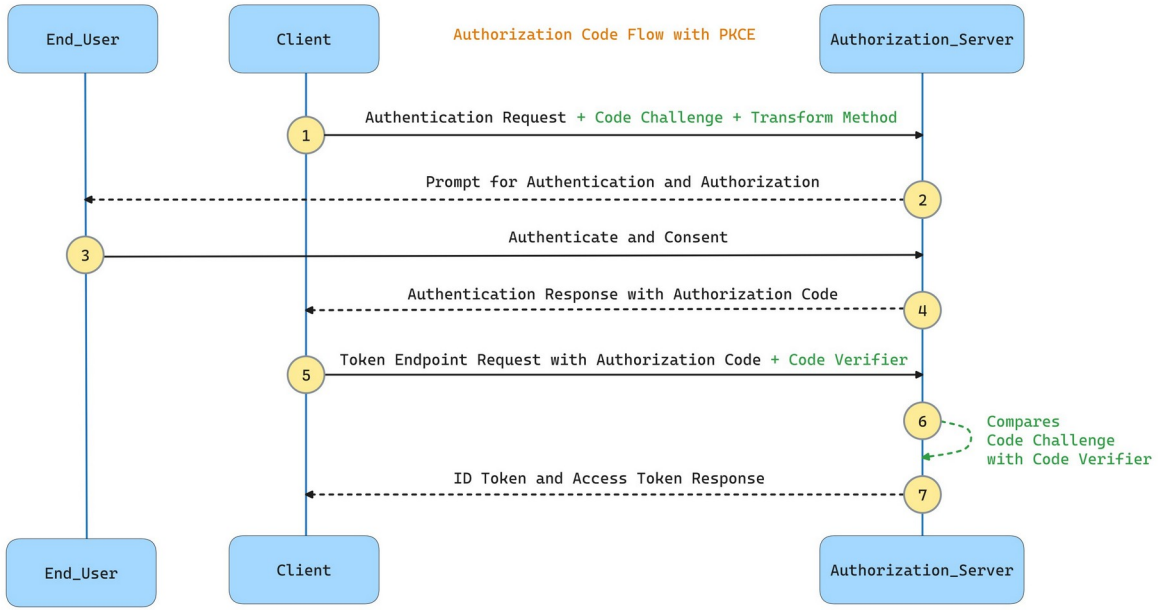


Image Credit: https://auth0.com/blog/pkce-in-web-applications-with-spring-security/

SPA

- Library choice
  - oidc-client-ts is the workhorse
    - It implements the full OIDC client spec
    - PKCE, token exchange, silent renewal, logout, ID token validation, JWT verification
    - Framework-agnostic; you could use it directly in any JavaScript application.
  - react-oidc-context is the React-specific wrapper
    - It takes oidc-client-ts and exposes its functionality as React Context and hooks
    - Components access auth state via useAuth() and don't have to know about the underlying client

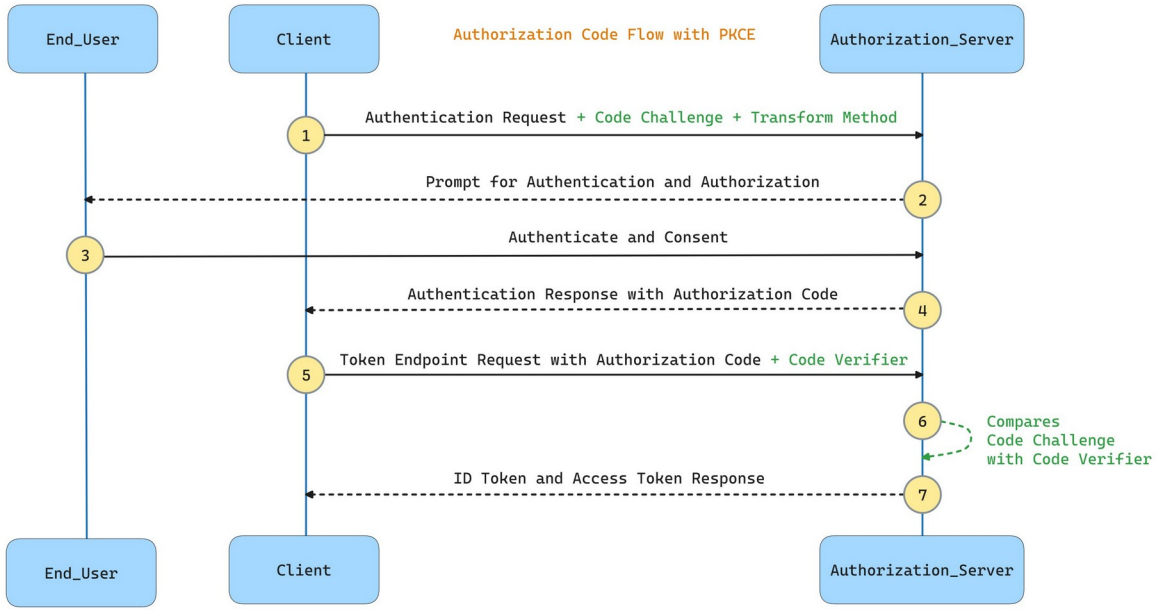


Image Credit: https://auth0.com/blog/pkce-in-web-applications-with-spring-security/

SPA

- Configuration
  - authority
    - The OIDC issuer; library fetches metadata from /.well-known/openid-configuration
  - client_id
    - Public identifier registered with the auth server
  - redirect_uri
    - Exact match required by the auth server for security
  - scope
    - What we're asking for
    - openid is required for OIDC
  - PKCE is on by default
    - No flag needed

```
import { AuthProvider } from 'react-oidc-context';

const oidcConfig = {
  authority: 'http://localhost:9000',          // mock auth server URL
  client_id: 'spa-client',                      // registered with the auth ser
  redirect_uri: 'http://localhost:5173/callback',
  post_logout_redirect_uri: 'http://localhost:5173/',
  response_type: 'code',                        // Authorization Code flow
  scope: 'openid profile email',                // OIDC scopes
  loadUserInfo: true,                           // fetch profile from /userinfo
};

function App() {
  return (
    <AuthProvider {...oidcConfig}>
      {/* rest of app */}
    </AuthProvider>
  );
}
```

SPA

- Configuration
  - authority
    - Is the URL where the OIDC provider lives
    - The library appends /.well-known/openid-configuration to fetch the provider's metadata
    - Endpoints, supported flows, signing keys. This means you don't hard-code endpoint URLs; the library discovers them.
  - client_id
    - Is a public identifier.
    - The auth server has a record of "this client_id is allowed, with these registered redirect URIs and scopes
  - redirect_uri
    - Is where the auth server sends the user back after login
    - The auth server validates this against its registered list and rejects anything that doesn't match exactly.

```
import { AuthProvider } from 'react-oidc-context';

const oidcConfig = {
  authority: 'http://localhost:9000',          // mock auth server URL
  client_id: 'spa-client',                      // registered with the auth ser
  redirect_uri: 'http://localhost:5173/callback',
  post_logout_redirect_uri: 'http://localhost:5173/',
  response_type: 'code',                        // Authorization Code flow
  scope: 'openid profile email',                // OIDC scopes
  loadUserInfo: true,                           // fetch profile from /userinfo
};

function App() {
  return (
    <AuthProvider {...oidcConfig}>
      {/* rest of app */}
    </AuthProvider>
  );
}
```

SPA

- Configuration
  - post_logout_redirect_uri
    - Is the analogous URL for after logout
    - The user lands here after the auth server's end-session flow completes
  - response_type:
    - 'code' selects the Authorization Code flow
    - Combined with the library's defaults, this means PKCE is automatically used
  - scope
    - Is what we're asking for
    - openid is required to make this an OIDC flow rather than a plain OAuth2 flow

```
import { AuthProvider } from 'react-oidc-context';

const oidcConfig = {
  authority: 'http://localhost:9000',          // mock auth server URL
  client_id: 'spa-client',                      // registered with the auth ser
  redirect_uri: 'http://localhost:5173/callback',
  post_logout_redirect_uri: 'http://localhost:5173/',
  response_type: 'code',                        // Authorization Code flow
  scope: 'openid profile email',                // OIDC scopes
  loadUserInfo: true,                           // fetch profile from /userinfo
};

function App() {
  return (
    <AuthProvider {...oidcConfig}>
      {/* rest of app */}
    </AuthProvider>
  );
}
```

SPA

- Configuration
  - loadUserInfo
    - Tells the library to call the auth server's /userinfo endpoint after login to fetch additional user details
    - Convenient for getting the user's name into the UI

```
import { AuthProvider } from 'react-oidc-context';

const oidcConfig = {
  authority: 'http://localhost:9000',          // mock auth server URL
  client_id: 'spa-client',                      // registered with the auth ser
  redirect_uri: 'http://localhost:5173/callback',
  post_logout_redirect_uri: 'http://localhost:5173/',
  response_type: 'code',                        // Authorization Code flow
  scope: 'openid profile email',                // OIDC scopes
  loadUserInfo: true,                           // fetch profile from /userinfo
};

function App() {
  return (
    <AuthProvider {...oidcConfig}>
      {/* rest of app */}
    </AuthProvider>
  );
}
```

AuthProvider

- <AuthProvider>
  - Wraps the app, manages auth state, exposes it via context
  - useAuth() is the hook every component uses to read auth state
  - Returns isAuthenticated, isLoading, error, user, and methods for signin/signout
  - The user object holds the ID token claims, access token, and profile
- Built on the pattern context + reducer, wrapped behind a hook

```
import { useAuth } from 'react-oidc-context';

function HeaderUserMenu() {
  const auth = useAuth();

  if (auth.isLoading) return <Spinner />;
  if (auth.error)     return <span>Error: {auth.error.message}</span>;

  if (auth.isAuthenticated) {
    return (
      <div>
        <span>{auth.user?.profile.name}</span>
        <button onClick={() => auth.signoutRedirect()}>Log out</button>
      </div>
    );
  }

  return <button onClick={() => auth.signinRedirect()}>Sign in</button>;
}
```

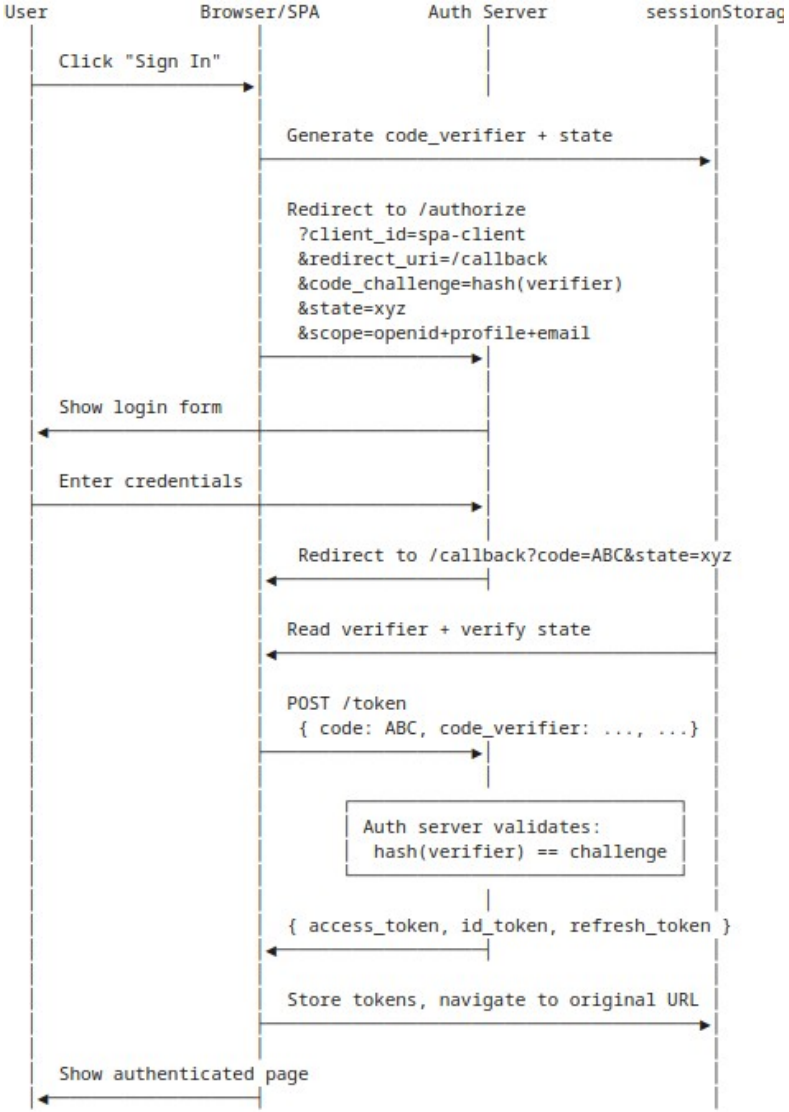
AuthProvider

- auth.user
  - When authenticated holds:
  - profile
    - Claims from the ID token and userinfo endpoint (name, email, sub, etc.)
  - access_token
    - The bearer token for API calls
  - id_token
    - The OIDC identity proof
  - expires_at
    - When the access token expires (Unix timestamp)

```
import { useAuth } from 'react-oidc-context';

function HeaderUserMenu() {
  const auth = useAuth();

  if (auth.isLoading) return <Spinner />;
  if (auth.error)     return <span>Error: {auth.error.message}</span>;

  if (auth.isAuthenticated) {
    return (
      <div>
        <span>{auth.user?.profile.name}</span>
        <button onClick={() => auth.signoutRedirect()}>Log out</button>
      </div>
    );
  }

  return <button onClick={() => auth.signinRedirect()}>Sign in</button>;
}
```

Login Flow

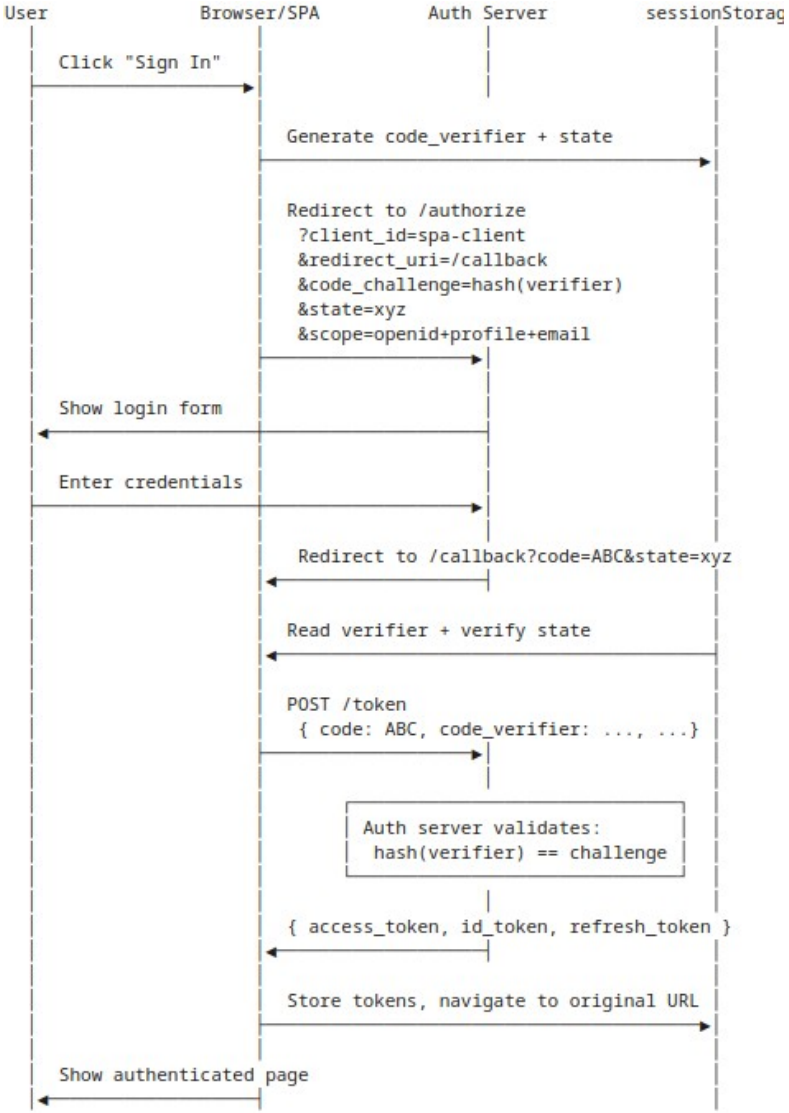
- Login Flow
  - The PKCE verifier stays in the browser the whole flow
  - The auth server never sees the verifier until the final exchange
  - The state parameter prevents code injection
    - The SPA verifies it on return
  - Steps with arrows to the auth server are real browser navigation
    - The SPA actually unloads
  - The library handles all of this
    - The SPA's only code is the <CallbackPage> and the Sign In button



Login Flow

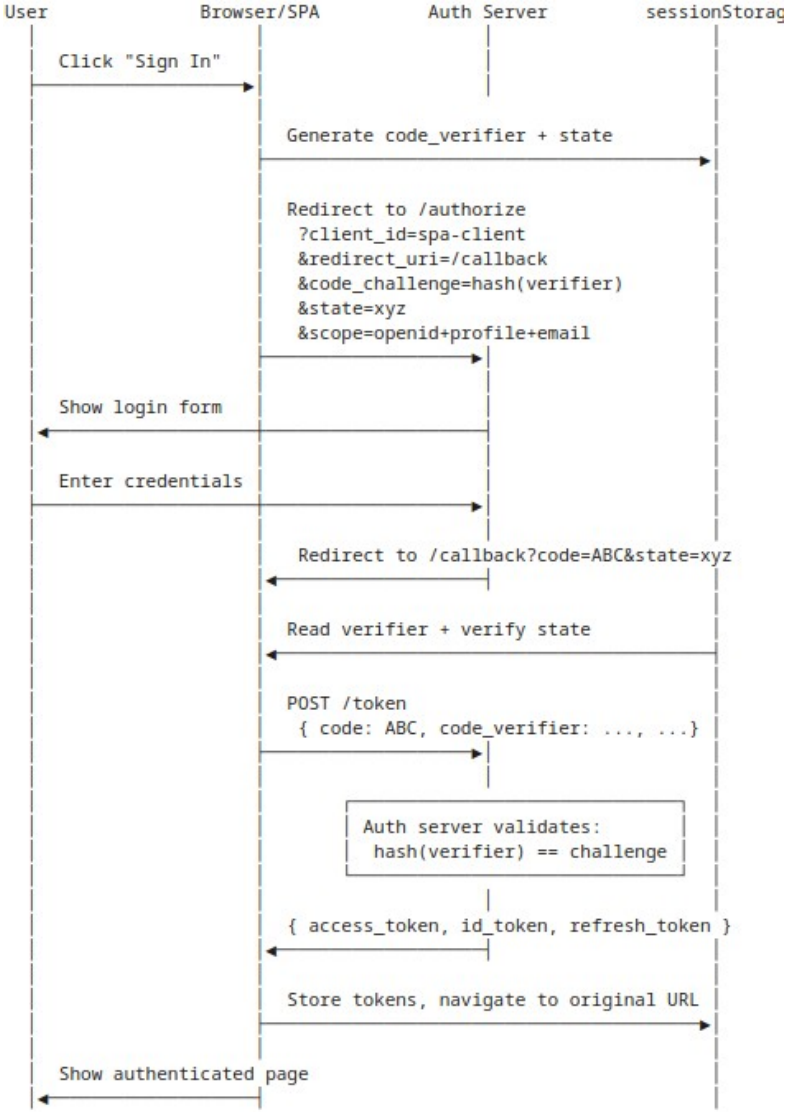
- Login Flow
  - Generate verifier
    - The library creates a random code_verifier string and computes

      code_challenge = SHA256(code_verifier)
    - The verifier goes into sessionStorage because the SPA is about to be unloaded by the redirect
    - Without sessionStorage, the verifier would be lost.
  - The /authorize redirect
    - This is a real browser navigation
    - The user's address bar changes to the auth server's URL
    - The SPA stops running
    - Any in-memory React state is destroyed.



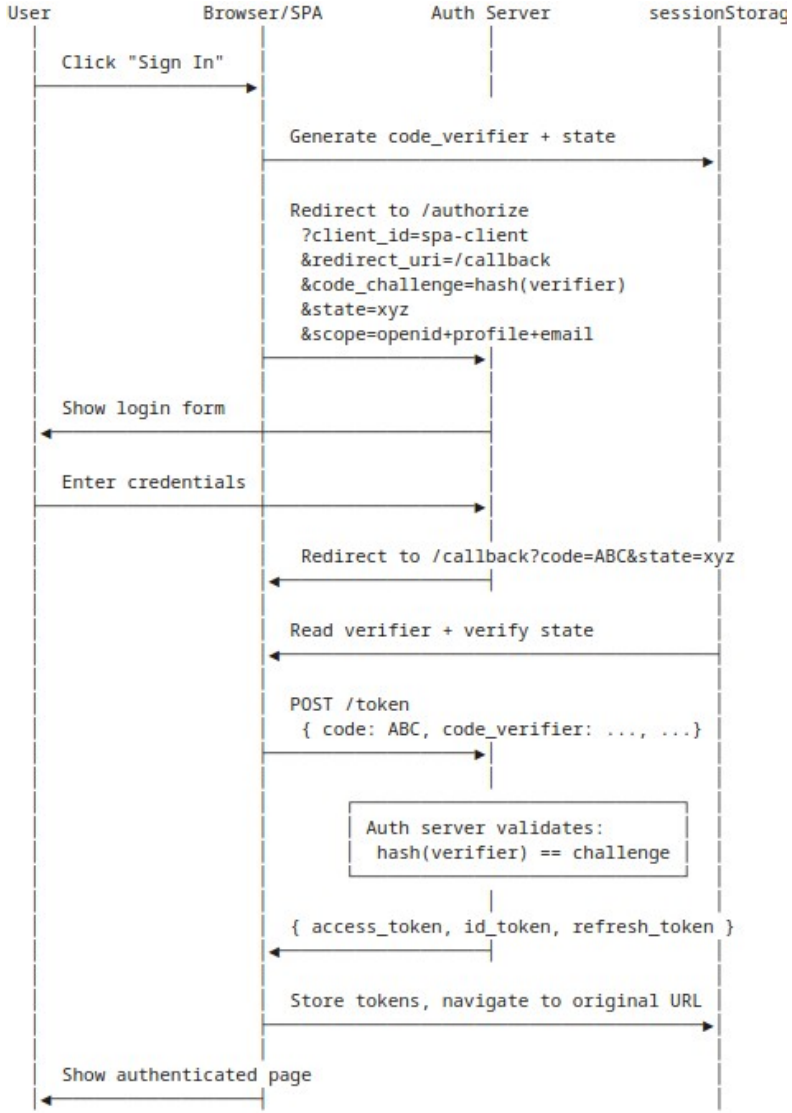
Login Flow

- Login Flow
  - User authenticates
    - Happens entirely on the auth server's pages
    - The SPA has no role here
    - This is also where the auth server may set its own session cookie so the user doesn't get re-prompted next time
  - The /callback redirect
    - The auth server sends the user back to the SPA with a fresh authorization code in the URL
    - The SPA loads from scratch
    - React Router renders the <CallbackPage>.



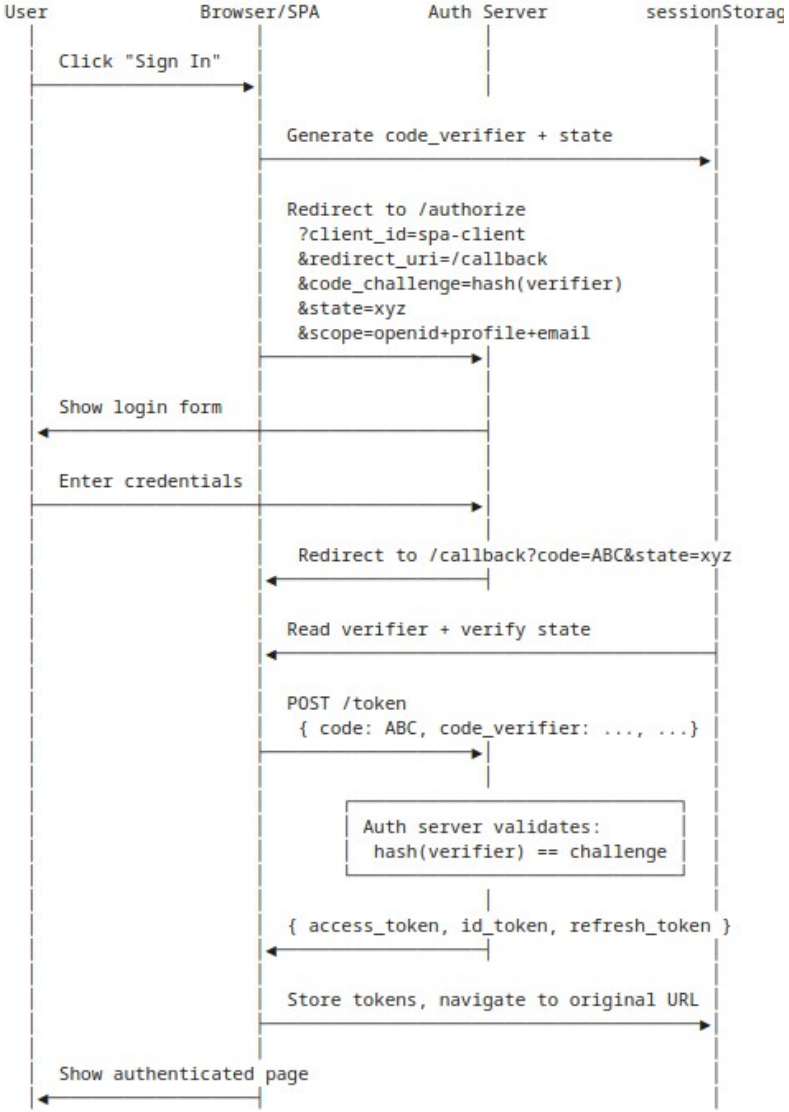
Login Flow

- Login Flow
  - Read verifier + verify state
    - The library retrieves the verifier from sessionStorage and checks the state parameter matches what was stored
    - If state doesn't match, this is a CSRF attack and the library aborts
  - POST /token
    - The actual code-for-token exchange
    - This is a background request, not a redirect
    - The SPA stays on the callback page
    - The verifier is sent here, completing PKCE.



Login Flow

- Login Flow
  - Tokens arrive
    - Access token, ID token, and (typically) a refresh token
    - The library stores them
    - Updates the auth context
    - And the <CallbackPage> navigates to the user's original destination



The Callback Page

- Callback page
  - The route the auth server redirects to after login
  - The library automatically detects it's on the callback URL
  - Performs the code-for-token exchange in the background
  - The component just shows a brief loading state and redirects when done
  - Use replace so the back button doesn't return to the callback URL

```
import { useAuth } from 'react-oidc-context';
import { Navigate } from 'react-router-dom';

function CallbackPage() {
  const auth = useAuth();

  if (auth.isLoading) return <div>Completing sign-in…</div>;
  if (auth.error)     return <div>Sign-in failed: {auth.error.message}</div>;
  if (auth.isAuthenticated) return <Navigate to="/" replace />;

  return <div>Completing sign-in…</div>;
}
```

The Callback Page

- What's actually happening
  - The auth server redirects to

    /callback?code=...&state=....
  - React Router renders <CallbackPage>
  - The <AuthProvider> detects on mount that the URL contains an auth response and starts the token exchange automatically.
  - While the exchange is in flight, auth.isLoading is true.
  - When tokens arrive, auth.isAuthenticated becomes true.
  - The component sees this and navigates to /.

```
import { useAuth } from 'react-oidc-context';
import { Navigate } from 'react-router-dom';

function CallbackPage() {
  const auth = useAuth();

  if (auth.isLoading) return <div>Completing sign-in…</div>;
  if (auth.error)     return <div>Sign-in failed: {auth.error.message}</div>;
  if (auth.isAuthenticated) return <Navigate to="/" replace />;

  return <div>Completing sign-in…</div>;
}
```

Protected Routes

- A small wrapper component
  - Checks auth state and either renders or redirects
  - Captures the originally-requested URL in the redirect's state
  - After login completes, the user is sent back where they were going
  - One pattern, applied to every protected route
  - This is the declarative version of route guards

```
import { Navigate, useLocation } from 'react-router-dom';
import { useAuth } from 'react-oidc-context';

function ProtectedRoute({ children }: { children: React.ReactNode }) {
    const auth = useAuth();
    const location = useLocation();

    if (auth.isLoading) return <Spinner />;

    if (!auth.isAuthenticated) {
        return <Navigate to="/login" state={{ from: location }} replace />;
    }

    return <>{children}</>;
}

// Used in the route table
<Route element={<AppLayout />}>
<Route path="/" element={<ProtectedRoute><HomePage /></ProtectedRoute>} />
<Route path="/accounts" element={<ProtectedRoute><AccountsPage /></ProtectedRoute>} />
    </Route>const AuthContext = createContext<{ state: AuthState; dispatch: Dispatch<AuthAction> } | null>(null);
```

Protected Routes

- A small wrapper component
  - The component is small but does meaningful work
  - Loading state
    - While the library is initializing or processing a redirect, render a spinner
    - Don't redirect yet; we don't know if the user is authenticated
  - Unauthenticated
    - Redirect to /login
    - Use state: { from: location } to remember where they were going
    - The replace prevents back-button bouncing between the login page and the protected page
  - Authenticated
    - Render the children.

```
import { Navigate, useLocation } from 'react-router-dom';
import { useAuth } from 'react-oidc-context';

function ProtectedRoute({ children }: { children: React.ReactNode }) {
    const auth = useAuth();
    const location = useLocation();

    if (auth.isLoading) return <Spinner />;

    if (!auth.isAuthenticated) {
        return <Navigate to="/login" state={{ from: location }} replace />;
    }

    return <>{children}</>;
}

// Used in the route table
<Route element={<AppLayout />}>
<Route path="/" element={<ProtectedRoute><HomePage /></ProtectedRoute>} />
<Route path="/accounts" element={<ProtectedRoute><AccountsPage /></ProtectedRoute>} />
    </Route>const AuthContext = createContext<{ state: AuthState; dispatch: Dispatch<AuthAction> } | null>(null);
```

The Login Page

- Login page
  - Shows the sign-in button, kicks off the OAuth flow
  - Reads from from the location state
    - Where the user was trying to go
  - Already authenticated
    - Redirect to from immediately (handles bookmarking)
  - Passes from along to signinRedirect so it survives the auth flow
  - Renders without the app's layout chrome
    - “Chrome” here means the stylistic embellishments on the page and not the browser

```tsx
import { useAuth } from 'react-oidc-context';
import { useLocation, Navigate } from 'react-router-dom';

function LoginPage() {
  const auth = useAuth();
  const location = useLocation();
  const from = (location.state as { from?: Location })?.from?.pathname ?? '/';

  if (auth.isAuthenticated) return <Navigate to={from} replace />;

  return (
    <div className="login-page">
      <h1>Sign in to continue</h1>
      <button onClick={() => auth.signinRedirect({ state: { from } })}>
        Sign in
      </button>
    </div>
  );
}
```

Silent Token Renewal

- Refreshing tokens
  - Access tokens expire
    - Typically 5 to 60 minutes
  - Refresh tokens let the SPA get a new access token without a user redirect
  - react-oidc-context handles renewal automatically when configured
  - Triggered proactively before expiry, or reactively on a 401
  - Refresh tokens should be rotated
    - Single-use, replaced on each refresh
  - If renewal fails, the user has to log in again

Behavior	`sessionStorage`	`localStorage`
User refreshes page	Stays logged in	Stays logged in
User opens app in a new tab	Logged out (must re-auth)	Logged in (tokens visible to new tab)
User closes browser, reopens tomorrow	Logged out	Logged in (until tokens expire)
User opens link from email	Logged out	Logged in

Silent Token Renewal

- The mechanics
  - The token response from initial login includes an access token (short-lived) and a refresh token (longer-lived)
  - Before the access token expires, the library calls the auth server's token endpoint with the refresh token
  - The auth server validates the refresh token and returns a new access token
  - The library updates its stored tokens and the SPA continues to work without the user noticing.

Behavior	`sessionStorage`	`localStorage`
User refreshes page	Stays logged in	Stays logged in
User opens app in a new tab	Logged out (must re-auth)	Logged in (tokens visible to new tab)
User closes browser, reopens tomorrow	Logged out	Logged in (until tokens expire)
User opens link from email	Logged out	Logged in

Logout

- Logging out
  - More subtle than it looks
  - Local logout
    - Clear the SPA's tokens, navigate to a logged-out state
  - OIDC RP-initiated logout
    - Also call the auth server's end-session endpoint
  - auth.signoutRedirect() does both
    - Clears local state, redirects to auth server logout, returns to post_logout_redirect_uri
  - Without RP-initiated logout, the user's auth server session is still active
  - Multi-tab sync
    - SessionStorage is per-tab; multiple tabs need separate logouts

```
src/
├── App.tsx                  ← <AuthProvider> wraps everything
├── routes/
│   ├── HomePage.tsx
│   ├── AccountsPage.tsx
│   ├── LoginPage.tsx        ← public — calls auth.signinRedirect()
│   └── CallbackPage.tsx     ← public — handles return from auth server
├── auth/
│   ├── ProtectedRoute.tsx   ← checks auth, redirects to /login
│   └── oidcConfig.ts        ← authority, client_id, scopes
├── api/
│   └── client.ts            ← Axios; will attach access_token (Unit 12)
└── main.tsx
```

BFF Pattern with Spring Boot

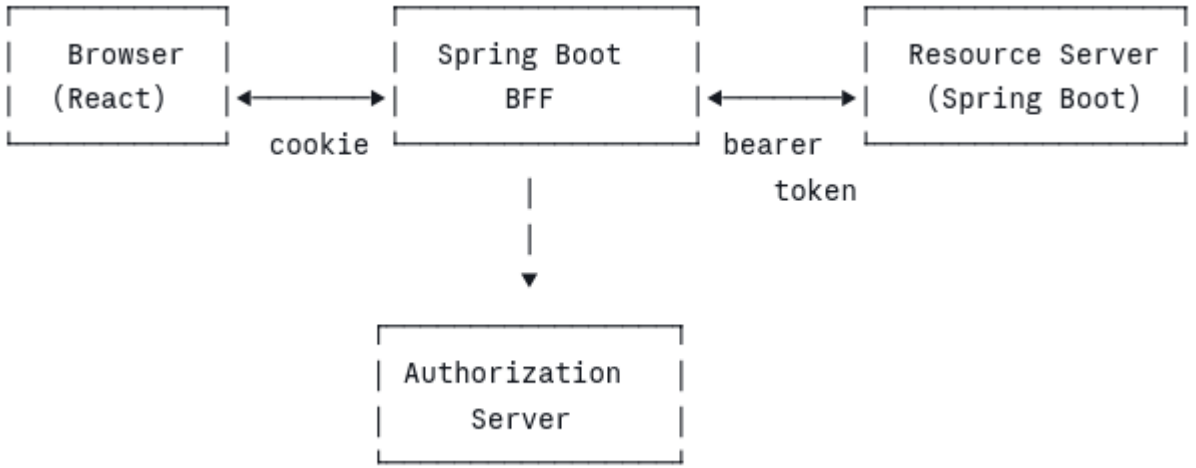
Pure-SPA Architecture

- The problem for pure-SPA
  - Tokens live in sessionStorage
    - Readable by any JavaScript on the origin
  - One XSS payload is a full account compromise
  - Mitigations help but don't eliminate the exposure
  - The architectural fix
    - Don't put tokens in JavaScript at all
    - But requires a backend component you control

```
src/
├── App.tsx                  ← <AuthProvider> wraps everything
├── routes/
│   ├── HomePage.tsx
│   ├── AccountsPage.tsx
│   ├── LoginPage.tsx        ← public — calls auth.signinRedirect()
│   └── CallbackPage.tsx     ← public — handles return from auth server
├── auth/
│   ├── ProtectedRoute.tsx   ← checks auth, redirects to /login
│   └── oidcConfig.ts        ← authority, client_id, scopes
├── api/
│   └── client.ts
└── main.tsx
```

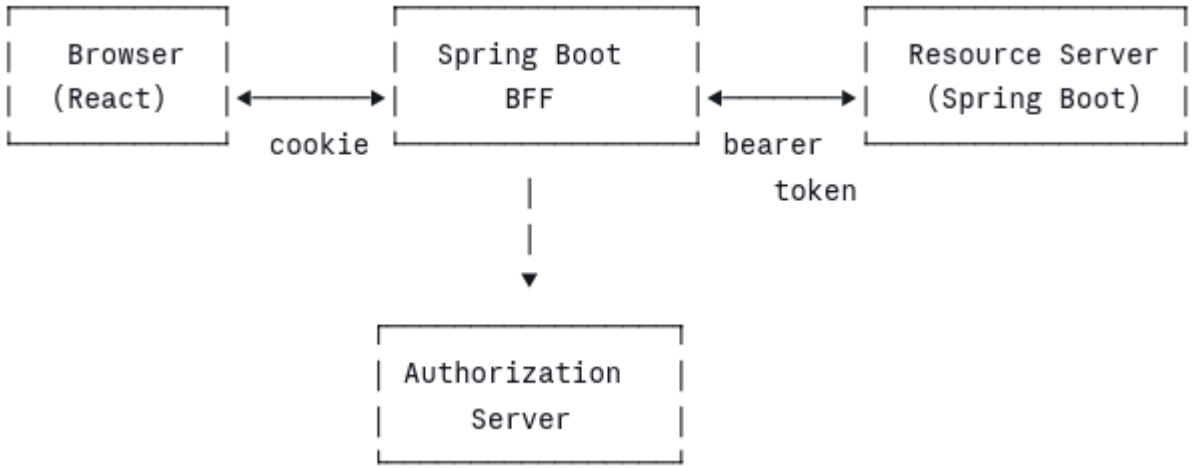
Three-Tier Architecture

- The BFF architecture
  - Browser
    - Holds only an HttpOnly session cookie; never sees a token
  - BFF
    - Spring Boot app; OAuth client; holds tokens server-side
  - Resource Server
    - The protected API
  - Authorization Server
  - The browser only ever talks to the BFF
  - The BFF talks to everything else



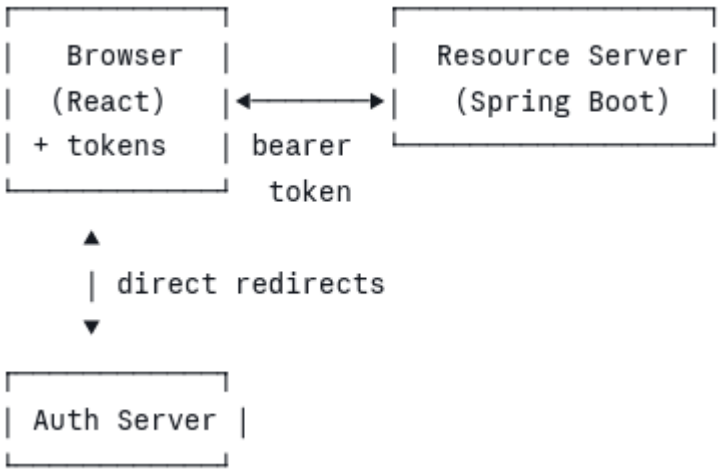
Three-Tier Architecture

- The BFF architecture
  - Browser interacts with BFF
    - Same-origin communication
    - Browser holds an HttpOnly session cookie issued by the BFF
    - Every request to the BFF includes that cookie automatically
    - JavaScript can't read or modify it



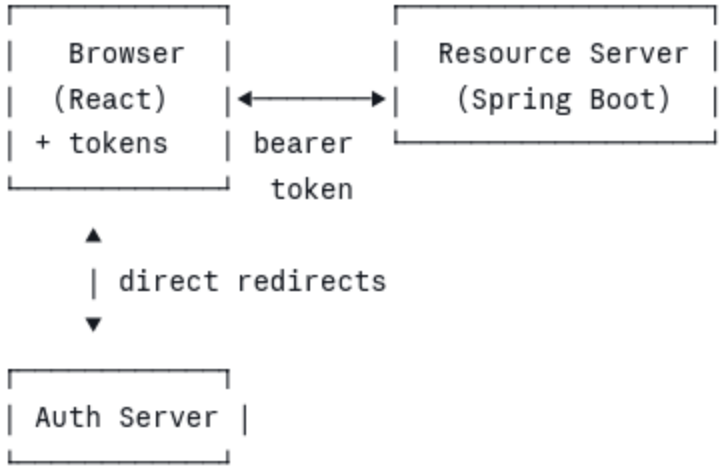
Three-Tier Architecture

- The BFF architecture
  - BFF interacts with Authorization Server
    - When a user clicks Sign In
    - The BFF, not the SPA, performs the OAuth Authorization Code + PKCE flow
    - The auth server's redirects go to the BFF, not to the browser-side SPA
    - The BFF receives the tokens directly from the token endpoint.



Three-Tier Architecture

- The BFF architecture
  - BFF interact with Resource Server
    - When the SPA needs data from the protected API, it makes a request to the BFF
    - The BFF looks up the user's stored tokens
    - Attaches the bearer token to an outbound request to the resource server
    - Gets the response, and proxies it back to the SPA.
  - The browser is only ever talking to one server (the BFF) over one credential (the session cookie)
  - Everything else happens server-side



What Spring Provides

- spring-boot-starter-oauth2-client provides
  - Full Authorization Code + PKCE flow implementation
  - OAuth2AuthorizedClient
    - Server-side token storage per user session
  - Automatic token refresh when the access token is near expiry

    /login/oauth2/authorization/{registrationId}
  - Auto-exposed login endpoint
  - Integration with Spring Security's session management
  - Configuration via application.yml
    - Most cases need no Java code

```yaml
spring:
  security:
    oauth2:
      client:
        registration:
          mock-auth:
            client-id: spa-client
            client-secret: ${OAUTH_CLIENT_SECRET}
            authorization-grant-type: authorization_code
            redirect-uri: "{baseUrl}/login/oauth2/code/{registrationId}"
            scope: openid, profile, email
        provider:
          mock-auth:
            issuer-uri: http://localhost:9000
```

What Spring Provides

- Config
  - registration.mock-auth
    - Defines a client; mock-auth is its registration ID
  - Client-secret
    - The BFF can have one; because it's a confidential client
  - Redirect-uri
    - Spring fills in the placeholders
    - resolves to http://localhost:8080/login/oauth2/code/mock-auth
  - scope
    - The OIDC scopes we're requesting
  - provider.mock-auth.issuer-uri
    - Spring auto-discovers endpoints from /.well-known/openid-configuration

```yaml
spring:
  security:
    oauth2:
      client:
        registration:
          mock-auth:
            client-id: spa-client
            client-secret: ${OAUTH_CLIENT_SECRET}
            authorization-grant-type: authorization_code
            redirect-uri: "{baseUrl}/login/oauth2/code/{registrationId}"
            scope: openid, profile, email
        provider:
          mock-auth:
            issuer-uri: http://localhost:9000
```

Auto-Exposed Endpoints

- Spring Security provides
  - /login/oauth2/authorization/mock-auth
    - Kicks off the OAuth flow
  - /login/oauth2/code/mock-auth
    - The OAuth callback URL
    - Registered with auth server
  - /logout
    - Logs out the user
    - Clears session
    - Can chain to auth server

```yaml
spring:
  security:
    oauth2:
      client:
        registration:
          mock-auth:
            client-id: spa-client
            client-secret: ${OAUTH_CLIENT_SECRET}
            authorization-grant-type: authorization_code
            redirect-uri: "{baseUrl}/login/oauth2/code/{registrationId}"
            scope: openid, profile, email
        provider:
          mock-auth:
            issuer-uri: http://localhost:9000
```

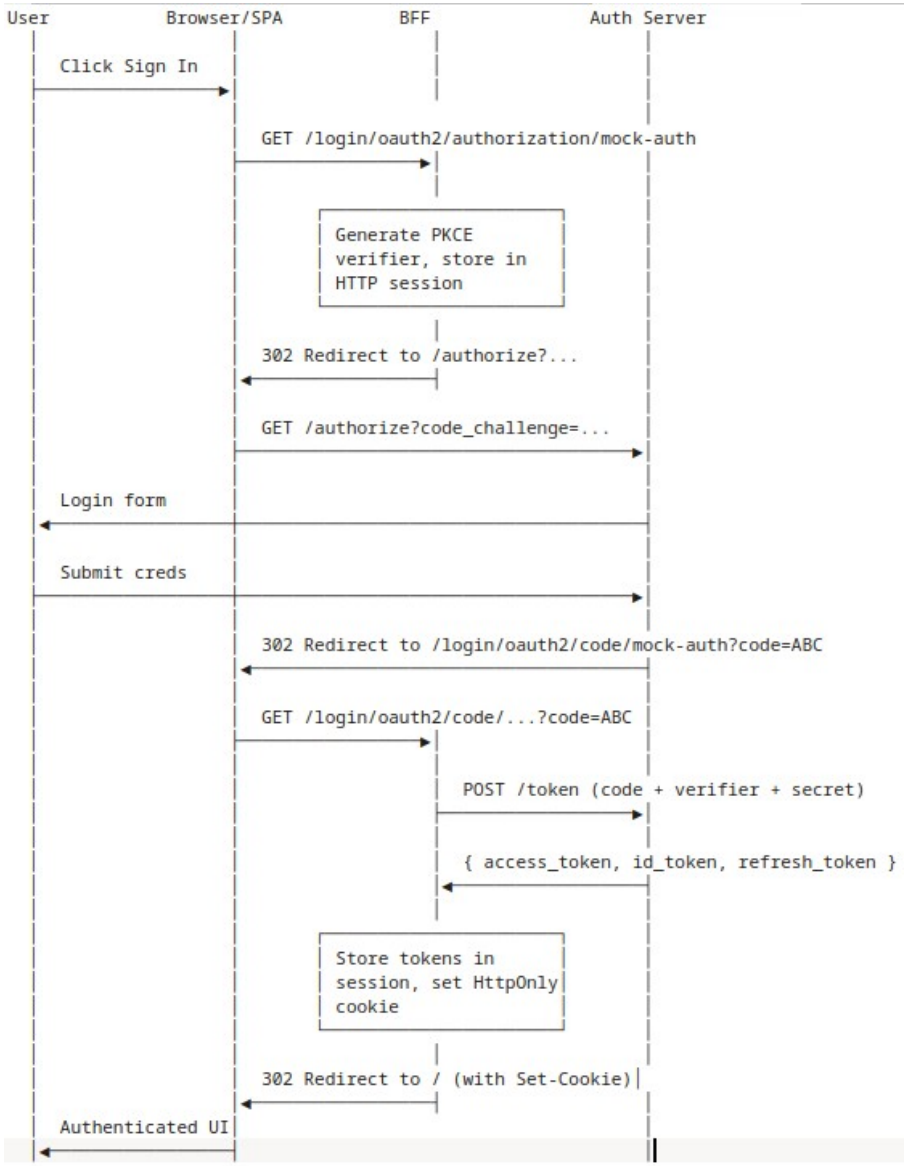
Auto-Exposed Endpoints

- Spring Security provides
  - CSRF protection
    - Enabled by default for cookie-authenticated apps
  - Session cookie
    - JSESSIONID (or whatever the container uses)
    - HttpOnly + Secure + SameSite by default

```yaml
spring:
  security:
    oauth2:
      client:
        registration:
          mock-auth:
            client-id: spa-client
            client-secret: ${OAUTH_CLIENT_SECRET}
            authorization-grant-type: authorization_code
            redirect-uri: "{baseUrl}/login/oauth2/code/{registrationId}"
            scope: openid, profile, email
        provider:
          mock-auth:
            issuer-uri: http://localhost:9000
```

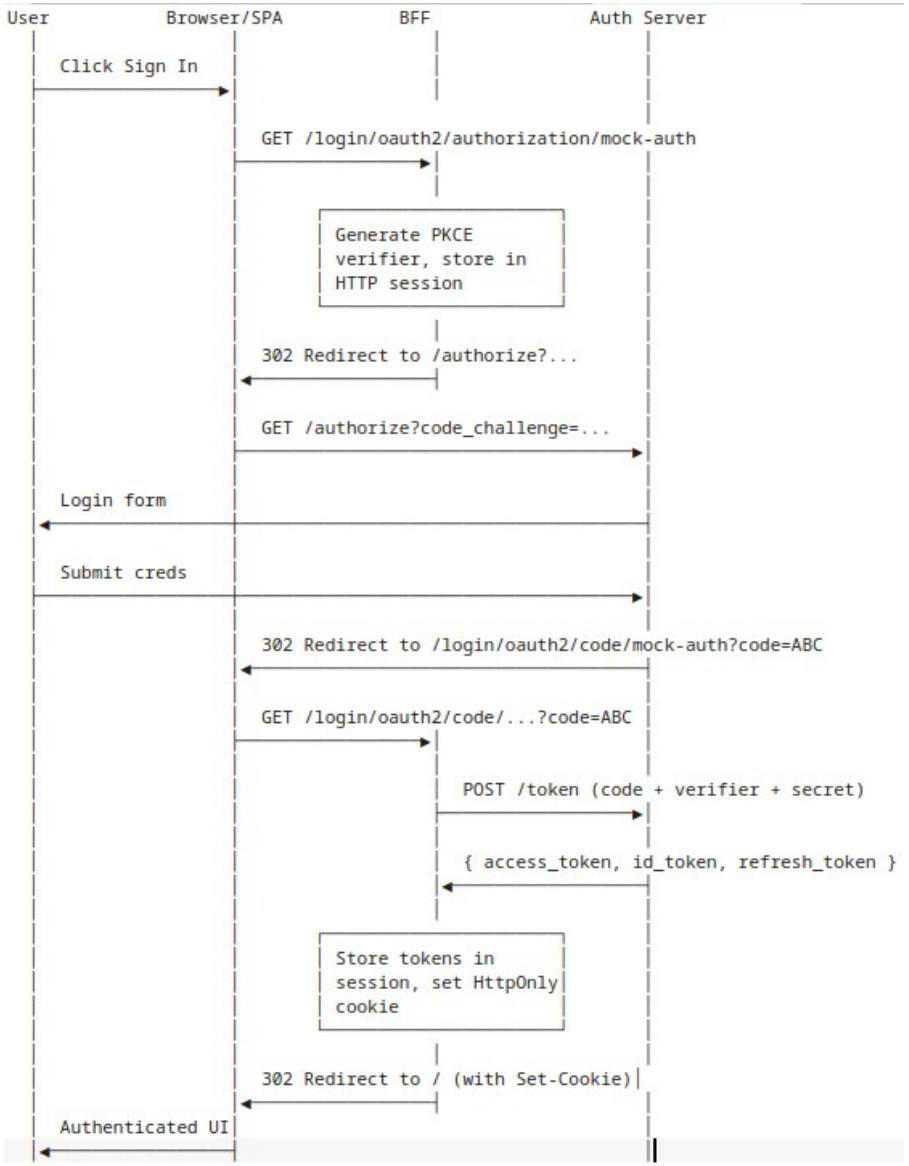
The Login Flow with a BFF

- Login flow
  - The SPA's only action is navigating to the BFF endpoint
    - No JavaScript library involved
  - PKCE verifier lives in the BFF's HTTP session, not in browser storage
  - Tokens never reach the browser
    - Only the session cookie does
  - Spring Security handles everything between the click and the final redirect



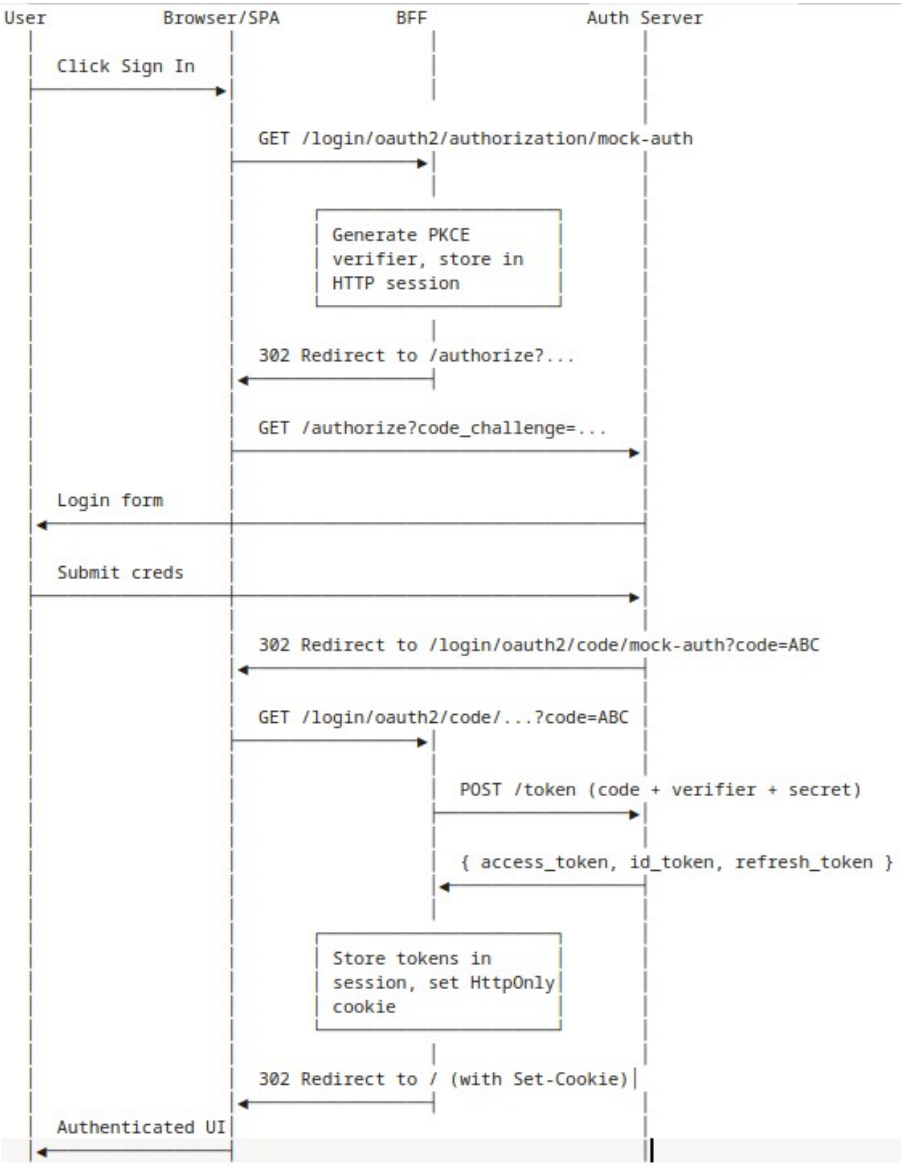
The Login Flow with a BFF

- The key differences from the pure-SPA flow
  - The SPA doesn't generate PKCE values
  - Spring does it server-side
  - The verifier never touches JavaScript.
  - The SPA doesn't handle the callback
    - The auth server redirects to a Spring endpoint, not a React route.
  - The SPA doesn't do the token exchange
    - Spring does it, with the client secret authenticating the BFF to the auth server.



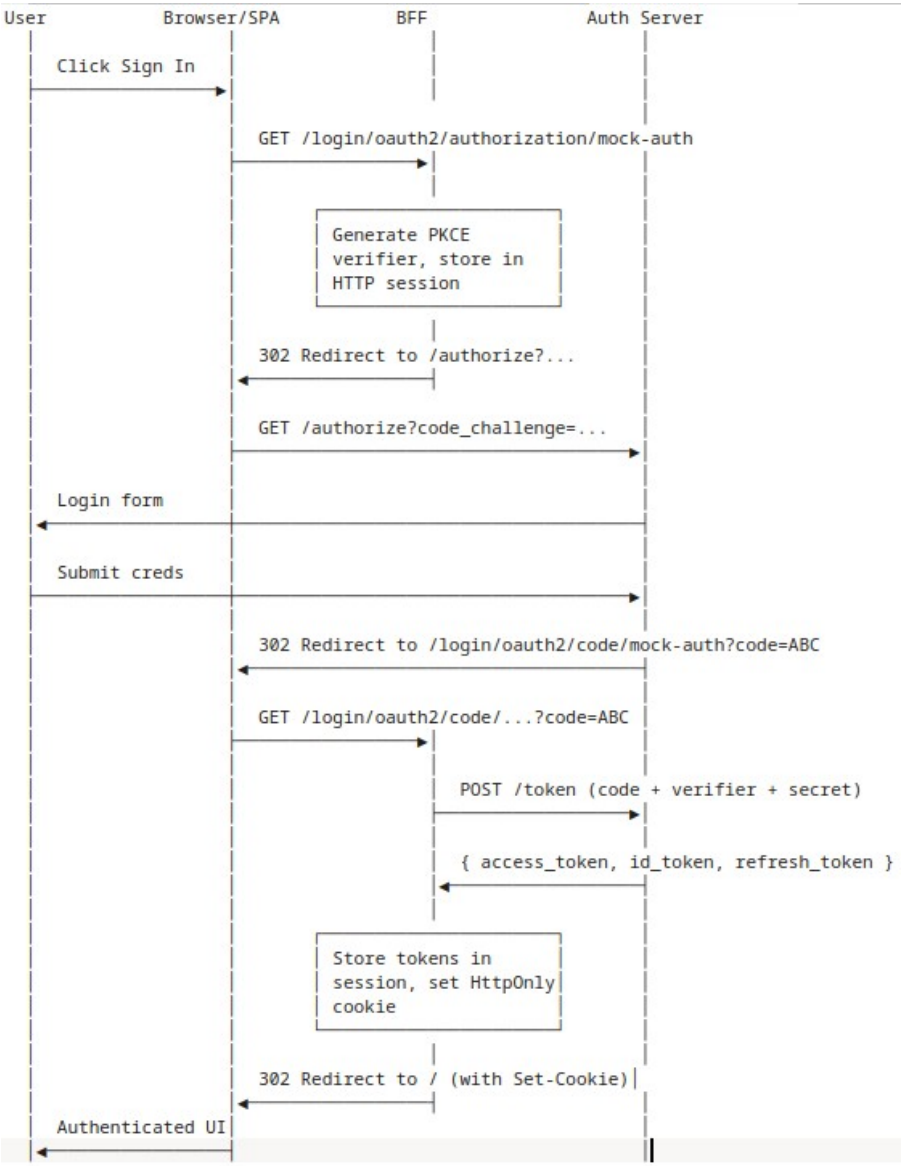
Same-Origin Deployment

- Why the SPA and BFF share an origin
  - The session cookie is scoped to the BFF's origin
  - For the cookie to be sent on every API call, the SPA must be on the same origin as the BFF
  - Two ways to achieve this
    - Spring Boot serves the SPA's static files alongside its endpoints
    - A reverse proxy (nginx, ingress) routes both /api/* and /* to the right backend
  - Side benefit
    - No CORS
    - Same-origin requests don't trigger CORS rules
  - Architectural simplification of BFF



The Browser

- What's in the browser after login
  - A single HttpOnly session cookie
    - JSESSIONID or similar
    - The cookie carries a session ID, not the tokens themselves
    - JavaScript cannot read the cookie, document.cookie doesn't show it
  - Tokens are at the BFF, keyed by session ID, never visible to the browser
  - An XSS attack can make requests while the page is open
    - But not exfiltrate the credential because the cookie can't be copied to another machine
    - Cannot act after the page closes. when the tab is gone, so is the attack



Resource Server

- The BFF's outbound half
  - Calling protected APIs
  - The BFF is also an OAuth client of the resource server
    - it attaches bearer tokens on outbound calls
    - Uses WebClient if reactive
    - Spring Security's interceptor transparently attaches the access token from the user's OAuth2AuthorizedClient
  - Automatic refresh
    - If the token is near expiry, the interceptor refreshes it first
  - Application code never touches tokens directly

```
@RestController
@RequestMapping("/api")
public class AccountsController {

    private final RestClient restClient;

    public AccountsController(RestClient.Builder builder,
                              OAuth2AuthorizedClientManager manager) {
        var oauth = new OAuth2ClientHttpRequestInterceptor(manager);
        oauth.setClientRegistrationIdResolver(req -> "mock-auth");
        this.restClient = builder.requestInterceptor(oauth).build();
    }

    @GetMapping("/accounts")
    List<Account> accounts() {
        return restClient.get()
            .uri("http://localhost:8081/accounts")
            .retrieve()
            .body(new ParameterizedTypeReference<>() {});
    }
}
```

Logout

- Logout is cleaner than pure SPA
  - POST /logout
    - Spring Security's built-in endpoint
    - Invalidates the session, clears the session cookie, removes the OAuth2AuthorizedClient
  - Tokens are wiped server-side
    - Nothing for an attacker to steal even if the browser is compromised
  - For full OIDC logout, configure OIDC end-session redirect
    - Spring redirects to the auth server's logout endpoint
  - Multi-tab logout is automatic
    - All tabs share the cookie; invalidating it logs out everywhere

```java
@RestController
@RequestMapping("/api")
public class AccountsController {

    private final RestClient restClient;

    public AccountsController(RestClient.Builder builder,
                              OAuth2AuthorizedClientManager manager) {
        var oauth = new OAuth2ClientHttpRequestInterceptor(manager);
        oauth.setClientRegistrationIdResolver(req -> "mock-auth");
        this.restClient = builder.requestInterceptor(oauth).build();
    }

    @GetMapping("/accounts")
    List<Account> accounts() {
        return restClient.get()
            .uri("http://localhost:8081/accounts")
            .retrieve()
            .body(new ParameterizedTypeReference<>() {});
    }
}
```

SPA

- No OAuth library
  - React-oidc-context is gone
  - No <AuthProvider>
    - There's no client-side auth state to manage
  - No <ProtectedRoute>
    - Spring Security gates the API
    - No token to check client-side
  - Login is a plain <a> link
    - Logout is a form post
- The auth complexity moved to Spring
  - The SPA got dramatically simple

```
// No more AuthProvider, no react-oidc-context
function App() {
    return (
        <BrowserRouter>
            <Routes>
                <Route element={<AppLayout />}>
    <Route path="/" element={<HomePage />} />
    <Route path="/accounts" element={<AccountsPage />} />
    </Route>
    </Routes>
    </BrowserRouter>
);
}

// Login button — just a link
function HeaderUserMenu({ user }: { user: User | null }) {
    if (!user) return <a href="/login/oauth2/authorization/mock-auth">Sign in</a>;

    return (
        <form method="POST" action="/logout">
    <input type="hidden" name="_csrf" value={getCsrfTokenFromCookie()} />
    <span>{user.name}</span>
    <button type="submit">Sign out</button>
    </form>
);
}
```

BFF

- Choosing the Architecture
  - BFF wins on security and session management
  - Pure SPA wins on operational simplicity and deployment flexibility
  - For enterprise apps with sensitive data, BFF is the current best practice
  - For public-facing low-stakes apps, pure SPA is still common and reasonable
  - The choice should be deliberate
    - Based on threat model, not just "what we know how to build"

Concern	Pure SPA	BFF
Backend required?	No	Yes
Token storage exposure	High (XSS = compromise)	Low (HttpOnly cookie)
Multi-tab logout	Manual sync needed	Automatic
Same-origin deployment	Optional	Required
CORS	Yes	No
Scaling concerns	Static-host CDN	Need session storage strategy
Operational complexity	Low	Moderate
Best for	Lower-stakes, no backend	Sensitive data, regulated industries

BFF

- The BFF wins:
  - Security
    - XSS exposure is dramatically smaller. For sensitive apps, this matters.
  - Operational simplicity at the auth layer
    - No silent renewal in the browser, no multi-tab sync, no client-side token storage decisions.
  - Single sign-on with other Spring services
  - If your enterprise has a Spring-heavy backend, putting your SPA in front of a Spring BFF means the auth integration is uniform

Concern	Pure SPA	BFF
Backend required?	No	Yes
Token storage exposure	High (XSS = compromise)	Low (HttpOnly cookie)
Multi-tab logout	Manual sync needed	Automatic
Same-origin deployment	Optional	Required
CORS	Yes	No
Scaling concerns	Static-host CDN	Need session storage strategy
Operational complexity	Low	Moderate
Best for	Lower-stakes, no backend	Sensitive data, regulated industries

SPA

- The pure SPA wins:
  - No backend to operate
    - For genuinely simple apps, this is a meaningful saving.
  - CDN deployment
    - A pure SPA can be served from edge locations with very low latency.
  - Independent scaling of frontend and backend
    - The SPA can be cached aggressivel
    - Only the API tier scales with load

Concern	Pure SPA	BFF
Backend required?	No	Yes
Token storage exposure	High (XSS = compromise)	Low (HttpOnly cookie)
Multi-tab logout	Manual sync needed	Automatic
Same-origin deployment	Optional	Required
CORS	Yes	No
Scaling concerns	Static-host CDN	Need session storage strategy
Operational complexity	Low	Moderate
Best for	Lower-stakes, no backend	Sensitive data, regulated industries

Questions?